中国研究生操作系统开源创新大赛

项目说明书

作品名称：面向智能机器人的 AIOS

参赛队伍：Xenoblade

完成日期：2026 年 10 月

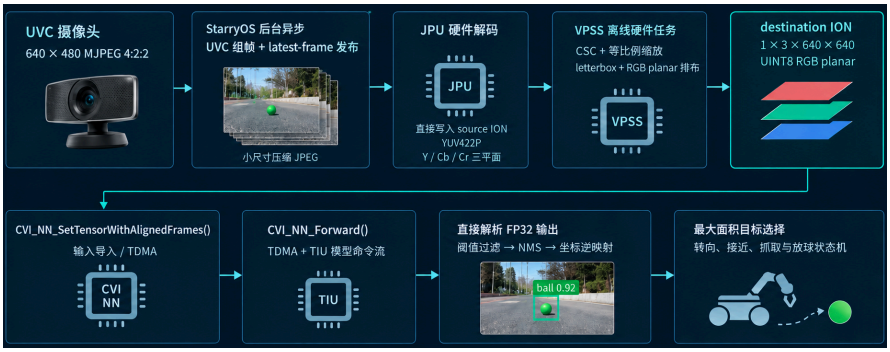
目 录

4 追球推理流程 **1**
4.1 JPU 硬件解码 1
4.2 VPSS 几何转换 1
4.3 TDMA 数据传输 2
4.4 TPU 硬件推理 2
4.5 运动控制追球 2
5 推理链路优化 **4**
5.1 优化前的处理链路与问题模型 5
5.2 异步采集与最新帧调度机制 6
5.3 JPU 硬件解码优化 8
5.4 ION 共享物理缓冲区 9
5.5 扩展 VPSS 驱动，直接接收 JPU 输出的 YUV422P 11
5.6 修复 VPSS 完成后的调度长尾 13
5.7 模型输入契约校验与正确的 TPU 输入导入 15
5.8 TPU ioctl ABI、ION 地址与完成语义修复 16
5.9 TPU 任务分类与四段时间戳计时机制 17
5.10 TPU 执行链路重构与调度开销消除 18
5.11 面向短时 TDMA 的中断唤醒与竞态管理 19
5.12 TPU 驱动优化 21
5.13 TPU 输出直读与轻量化检测后处理 22
5.14 最终总结 24
6 机器人用户态程序优化 **25**
6.1 基于视觉尺度反馈的二次曲线渐进接近控制 25
6.2 可识别红色容器 27
6.3 摄像头完全遮挡后的脱困与视觉恢复 28
6.4 多目标识别与基于视觉距离的抓取排序 29
7 StarryOS 与 Linux 视觉性能对比分析 **31**
7.1 测试结果概述 31
7.2 各项指标的含义以及计算方式 31
7.3 表格结果解释 34
7.4 StarryOS 快于 Linux 的原因分析 34
7.5 StarryOS 慢于 Linux 的可能原因 35
7.6 对置信度结果的解释 37
7.7 综合结论 37
附录 **38**
当前正式链路的函数级调用过程 38
初始化阶段 38
后台 UVC 采集阶段 38

前台摄像头与 JPU 阶段 . . . 38
20.4 VPSS 阶段 . . . 39
20.5 TPU 输入与模型前向阶段 . . . 39
20.6 后处理与控制阶段 . . . 40
22.3 强制 JPU 输出 NV12 . . . 40
22.4 普通模型输入直接物理地址绑定 . . . 40
22.5 P2 aligned 640 模型 . . . 40
22.7 固定跳帧或降低处理帧数 . . . 40
22.8 继续提高 compute 线程优先级 . . . 40

4 追球推理流程

本系统面向 SG2002 平台构建了一条由硬件加速器协同完成的网球视觉推理链路，整体采用“异步采集—硬件解码—硬件预处理—神经网络推理—检测后处理—运动控制”的处理流程。摄像头通过通用串行总线（Universal Serial Bus，USB）视频设备类协议（USB Video Class，UVC）输出分辨率为 640×480 的码流；该码流由 JPEG 压缩图像连续组成，格式称为 MJPEG（Motion JPEG）。StarryOS 内核中的后台采集任务持续接收并组装 UVC 数据包，使用单调递增的帧序号发布最新完整帧。前台视觉线程只获取序号更新的图像；当图像采集速度高于推理速度时，系统允许旧帧被新帧覆盖，从而避免历史图像在队列中积压，保证机器人使用尽可能接近当前环境状态的图像进行闭环控制。



4.1 JPU 硬件解码

获得最新 MJPEG 帧后，系统使用 JPEG 处理单元（JPEG Processing Unit，JPU）完成硬件解码。当前摄像头的 JPEG 图像采用 4:2:2 色度采样，因此 JPU 将其解码为 640×480 的亮度—色度 4:2:2 平面格式（YUV 4:2:2 Planar，YUV422P），并通过直接存储器访问（Direct Memory Access，DMA）将亮度 Y、色度 Cb 和色度 Cr 三个平面直接写入 ION 内存分配器提供的共享缓冲区；ION 是该分配器的专有名称。该缓冲区同时具有中央处理器（Central Processing Unit，CPU）虚拟地址和硬件可访问的物理地址，可被后续视频处理模块直接使用，避免将整幅 YUV 图像复制到普通用户态临时内存。针对摄像头偶发的损坏 JPEG，驱动还实现了异常帧恢复机制；当 JPU 出现码流耗尽、超时或输出长度异常时，系统会丢弃当前帧、复位解码器并等待更新帧，而不会因单帧错误导致视觉线程永久阻塞。

4.2 VPSS 几何转换

JPU 解码完成后，YUV422P 图像被送入视频处理子系统（Video Processing Subsystem，VPSS）执行硬件预处理。VPSS 在一次离线任务中完成颜色空间转换（Color Space Conversion，CSC）、等比例几何变换、letterbox 填充以及红绿蓝（Red, Green, Blue，RGB）三平面排布。对于 640×480 的摄像头图像和 640×640 的模型输入，原始 4:3 图像保持纵横比并置于 640×640 画布中央，上下区域使用填充值补齐，从而避免直接拉伸造成网球外观畸变。处理结果以 8 位无符号整数（Unsigned 8-bit Integer，UINT8）形式存放在目标 ION 缓冲区中，并按照批次、通道、高度、宽度（Number, Channel, Height, Width，NCHW）布局组织为 `1×3×640×640` 的 RGB planar 数据。由于 VPSS 输出格式与模型输入契约一致，系统无需再由 CPU 执行逐像素 YUV 到 RGB 转换、图像缩放、通道重排或浮点归一化。

4.3 TDMA 数据传输

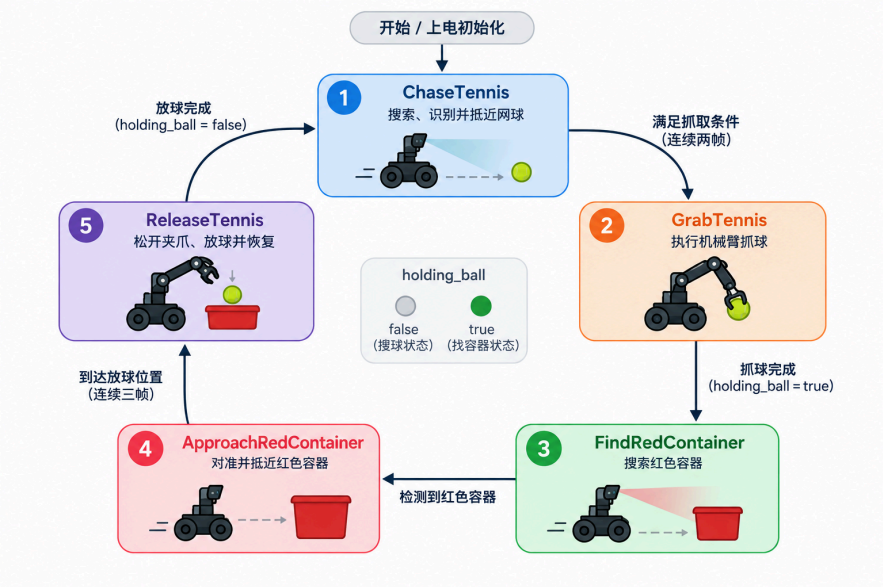
模型推理阶段使用 YOLOv8 网球检测模型。VPSS 生成的 RGB planar 物理缓冲区首先通过张量直接存储器访问（Tensor Direct Memory Access，TDMA）导入模型输入张量，随后由 CVITEK/SOPHGO 神经网络运行时调用 `CVI_NN_Forward()` 执行前向推理。运行时将模型编译生成的 TDMA 和张量指令单元（Tensor Instruction Unit，TIU）命令提交给张量处理器（Tensor Processing Unit，TPU）驱动：TDMA 负责输入、权重、特征图和中间张量的数据传输，TIU 负责卷积、激活及其他张量运算。当前 TPU 工作频率为 700 MHz，片上互连域频率为 900 MHz。任务触发后，提交线程进入等待状态，并由中断请求（Interrupt Request，IRQ）在硬件执行完成时唤醒；驱动同时保留内存映射输入/输出（Memory-Mapped Input/Output，MMIO）轮询作为异常情况下的兜底机制。经过调度优化后，推理任务由提交线程直接串行执行，消除了旧实现中提交线程、TPU 工作线程和完成等待线程之间的多次调度交接。内核分段测试表明，当前约 39～41 ms 的 TPU 推理耗时主要来自指令触发至硬件中断之间的真实网络执行，而不是软件排队延迟。

4.4 TPU 硬件推理

TPU 前向计算完成后，用户态直接解析运行时管理的 32 位浮点数（32-bit Floating Point，FP32）输出张量。程序按照 YOLOv8 的通道优先布局读取每个候选框的中心坐标、宽度、高度和网球类别置信度，并首先依据置信度阈值剔除低质量候选框；当前默认阈值为 0.5。随后采用非极大值抑制（Non-Maximum Suppression，NMS）处理重叠候选框：程序按置信度降序排列候选框，并利用交并比（Intersection over Union，IoU）判断不同候选框的空间重叠程度，当 IoU 大于 0.5 时抑制置信度较低的重复框。由于模型输入图像经过 letterbox 处理，NMS 后还需要根据缩放比例和上下填充量执行坐标逆映射，将预测框从 640×640 模型坐标系恢复到 640×480 摄像头坐标系，为后续运动决策提供真实图像位置。

4.5 运动控制追球

运动控制层从经过阈值过滤和 NMS 的检测结果中选择面积最大的网球框作为当前追踪目标，以降低远处小目标对近距离抓取任务的干扰。系统使用检测框水平中心坐标表示网球相对于夹爪中心线的方向偏差，并以检测框面积占整幅图像面积的比例近似描述网球距离。当未检测到网球时，机器人执行原地旋转搜索；检测到远距离网球后，根据目标位于期望中心的左侧、右侧或中心区域，分别执行左转、右转或直行接近。目标接近抓取区域后，系统扩大中心容差并缩短转向脉冲，减少离散视觉反馈造成的左右过冲。当检测框面积比例达到 0.55、目标中心进入夹爪有效区域且连续两帧确认成立时，底盘停止并触发原有机械臂抓取序列。若网球面积比例达到 0.85，系统将其判定为距离过近，先执行轻微后退再开始抓取。由此，系统实现了从图像采集、硬件推理、空间位置估计到车辆转向、接近和机械臂抓取的完整视觉闭环。


开始 / 上电初始化
1
ChaseTennis
搜索、识别并抵近网球
满足抓取条件
(连续两帧)
2
GrabTennis
执行机械臂抓球
抓球完成
(holding_ball = true)
3
FindRedContainer
搜索红色容器
检测到红色容器
4
ApproachRedContainer
对准并抵近红色容器
到达放球位置
(连续三帧)
5
ReleaseTennis
松开夹爪、放球并恢复
放球完成
(holding_ball = false)
holding_ball
false
(搜球状态)
true
(找容器状态)

5 推理链路优化

本文讨论的正式视觉链路以当前 640×640 网球模型为准。输入摄像头输出 640×480、Motion JPEG（MJPEG，运动 JPEG）压缩图像，JPEG 图像的亮度分量采样因子为 `0x21`，即 4:2:2 色度采样。模型输入张量为 `1×3×640×640`、8 位无符号整数、RGB planar、NCHW（Number–Channel–Height–Width，批次—通道—高度—宽度）布局；模型输出张量为 `[1,5,8400,1]` 的 32 位浮点数。正式模型文件为 `model/yolov8n_tennis_v2.cvimodel`.

缩写、数据格式与关键概念

缩写或术语	全称	本项目中的含义
USB	Universal Serial Bus	摄像头与 SG2002 之间的数据总线
UVC	USB Video Class	USB 摄像头的视频类协议及数据包格式
MJPEG	Motion Joint Photographic Experts Group	摄像头连续输出独立 JPEG 图像帧的压缩视频格式
JPU	JPEG Processing Unit	SG2002 片上 JPEG 硬件解码单元
YUV422P	YUV 4:2:2 Planar	Y、Cb、Cr 三平面存储，水平方向色度二分之一采样
VPSS	Video Processing Subsystem	完成颜色空间转换、缩放、边框填充和输出排布的硬件模块
CSC	Color Space Conversion	YUV 与 RGB 等颜色空间之间的转换
DMA	Direct Memory Access	硬件在不经 CPU 逐字节复制的情况下访问内存
ION	专有分配器名称	为多媒体与 TPU 提供物理连续、设备可访问的共享内存
TDMA	Tensor Direct Memory Access	TPU 的张量数据搬运单元
TIU	Tensor Instruction Unit	TPU 中执行卷积、激活等张量运算的计算单元
TPU	Tensor Processing Unit	SG2002 神经网络硬件加速器
IRQ	Interrupt Request	硬件完成后通知 CPU 的中断请求
MMIO	Memory-Mapped Input/Output	通过内存映射寄存器控制设备
ABI	Application Binary Interface	用户态与内核态之间稳定的数据结构、命令号和返回语义
ioctl	Input/Output Control	用户态通过文件描述符向设备驱动提交控制请求的接口
NCHW	Number–Channel–Height–Width	模型输入的四维张量排列顺序
NMS	Non-Maximum Suppression	去除同一目标重复检测框的非极大值抑制算法
IoU	Intersection over Union	两个检测框交集面积与并集面积之比
latest-frame	最新帧语义	消费者只处理最新完整帧，允许淘汰过时帧而不积压历史队列
letterbox	等比例缩放加边框填充	保持原图纵横比，将剩余区域填充成模型输入尺寸
zero-copy	零拷贝	大尺寸解码图像在设备间共享同一物理缓冲区；本项目并非所有数据完全无复制
W1C	Write One to Clear	向状态位写入 1 才能清除的中断寄存器语义

5.1 优化前的处理链路与问题模型

早期处理流程可概括为：前台应用同步等待摄像头图像，将压缩图像或 YUV 图像取回普通用户态内存，由中央处理器完成 JPEG 解码后的颜色转换、缩放、通道重排和 letterbox，再把处理结果复制到 TPU 输入张量，调用模型前向推理，最后在 CPU 上进行候选框解析与 NMS。该流程能够完成功能验证，但没有充分使用 SG2002 的 TPU, JPU 和 VPSS，也没有形成适合实时闭环控制的缓冲与调度模型。

早期观测数据为：视频采集约 73 ms、CPU 预处理约 56 ms、TPU 推理约 56 ms、后处理约 14 ms，单帧总处理时间约 189 ms。需要说明的是，四个分项直接相加为 199 ms，因此该早期数据只能作为瓶颈量级证据，不能与后续精确分段计时进行严格算术对比。不过它足以说明三个结构性问题：第一，采集是同步阻塞的；第二，约 56 ms 的像素级预处理消耗了有限的 C906 CPU 资源；第三，图像在普通缓冲区、设备缓冲区和 TPU 张量之间存在多次大块复制。单纯提高某个模块的吞吐量不一定缩短“看到目标—产生控制决策”的响应时间。例如，摄像头可以持续产生 30 帧每秒，但若应用严格按照先进先出顺序处理，每帧推理耗时又高于帧周期，队列就会积压，机器人实际处理的是越来越旧的图像。类似地，VPSS 硬件实际只运行 1.5 ms，但若等待线程在完成中断后 40 ms 才重新获得 CPU，则用户观察到的 VPSS 时间仍接近 42 ms。因此，优化必须同时覆盖数据面和控制面：

1. 数据面优化关注 JPEG、YUV 和 RGB 数据如何在 UVC、JPU、VPSS 与 TPU 之间流动，目标是减少 CPU 像素计算与大图复制；
2. 控制面优化关注任务如何入队、等待、被中断唤醒和恢复执行，目标是消除排队、错误唤醒策略和不必要的线程交接；
3. 契约优化关注每个设备真实支持的像素格式、步长、地址范围和张量布局，目标是在加速的同时保持检测结果正确；
4. 可观测性优化关注时间戳边界和异常统计，目标是区分硬件慢、调度慢、相位等待和统计重复；
5. 稳定性优化关注损坏 JPEG、位流耗尽、超时、摄像头复位以及错误码语义，目标是避免单帧故障演化成整机卡死。

最终正式链路如下：



该架构的核心不是把所有工作都放进内核，而是在最适合的位置完成每项工作：内核负责设备生命周期、物理缓冲区访问、异步采集、硬件提交、中断和异常恢复；用户态负责模型契约校验、设备组合、检测后处理和机器人策略。这样既保留了策略迭代的灵活性，也避免让用户态重复实现硬件驱动职责。

5.2 异步采集与最新帧调度机制

原始实现采用同步采集模式，视觉线程只有在需要执行推理时才向摄像头请求图像，并持续阻塞至通用串行总线视频类（USB Video Class，UVC）数据包被组装为一幅完整图像。在这种模式下，图像采集、JPEG 解码、图像预处理和张量处理器（Tensor Processing Unit，TPU）推理严格串行，摄像头无法在视觉线程执行推理期间准备下一帧。如果请求时刻刚好错过上一幅图像，视觉线程还可能等待接近一个完整的摄像头帧周期。若使用先进先出队列保存所有采集帧，当图像生产速度高于推理速度时，又会形成历史帧积压，使机器人基于过时图像进行运动决策。因此，本系统没有追求处理摄像头产生的每一帧，而是以降低图像帧龄为目标，使每次视觉决策尽可能使用最新的完整图像。



为此，摄像头驱动在 `cvi_usb_camera.rs` 中引入了 `AsyncCapture` 和 `LatestFrameStore`。`AsyncCapture` 负责管理后台采集任务的启动、停止、等待和消费状态。用户态通过 `CVI_CAMERA_IOCTL_START_ASYNC` 启动后台采集线程，驱动会检查其运行状态，避免重复启动多个线程并发访问同一个 UVC 端点。设备关闭或用户态发出 `CVI_CAMERA_IOCTL_STOP_ASYNC` 请求时，驱动设置停止标志并唤醒相关等待者，使后台任务能够有序退出。这样，图像采集从前台视觉线程中分离出来，使摄像头可以在前台执行解码、预处理和推理时准备下一幅图像。

`LatestFrameStore` 采用双缓冲机制保存 MJPEG 图像。两个 `Vec<u8>` 缓冲区分别承担“已发布缓冲区”和“生产者写入缓冲区”的角色：消费者只能读取当前已发布缓冲区，而后台线程始终向另一个非活动缓冲区写入新数据。其写入索引可以表示为 `published_index XOR 1`。只有当一幅 JPEG 图像完成组装并通过完整性检查后，驱动才统一更新帧元数据、递增帧序号并切换已发布索引。在此之前，消费者仍然可以安全读取上一幅完整图像。因此，双缓冲将正在接收的可变数据与已经发布的稳定快照分离，既防止消费者读取到半帧，也避免在整个 USB 数据包组装过程中长时间持有互斥锁。每一幅成功发布的图像都带有单调递增的帧序号 `sequence`。

为了避免后台采集线程持续占用单核 C906，当前实现还增加了“一帧预取”背压机制。后台任务只在尚未发布任何图像，或者消费者已经通过 `mark_consumed(sequence)` 确认取得当前图像后，才开始下一次采集。因此，该机制不是允许生产者无限超前运行的无界异步流水线，而是容量上限为一幅已发布图像的有界生产者—消费者协议。用户态取得一幅有效图像后，驱动更新 `last_consumed_sequence` 并唤醒生产者；生产者随后可以在前台处理当前图像期间采集下一帧。该设计既实现了图像采集与视觉处理的前后台重叠，又限制了 UVC 线程对 USB 主机栈和中央处理器（Central Processing Unit，CPU）资源的持续占用，同时避免形成历史帧队列。完整图像在发布前还需要经过 JPEG 完整性校验。`capture_frame()` 最多执行八次底层采集，每次调用 `uvc_capture_one_frame_profiled()` 后，都会检查图像开头是否包含 JPEG 图像起始标志（Start of Image，SOI）`0xFF 0xD8`，末尾是否包含图像结束标志（End of Image，EOI）`0xFF 0xD9`，以及图像长度是否达到 `MIN_VALID_JPEG_BYTES=4096`。任一条件不满足时，驱动分别累计 `invalid_soi`、`invalid_eoi` 或 `invalid_too_small`，同时更新总无效帧数和重试次数，并重置 UVC 帧标识符（Frame Identifier，FID）的连续性状态。这样可以防止损坏帧的残留包状态影响下一次组帧。只有连续八次采集均失败时，驱动才向上层返回错误。

经过上述改造，摄像头子系统形成了由后台异步采集、JPEG 双缓冲、帧序号同步、最新帧发布、一帧背压和异常帧恢复共同组成的完整机制。其中，双缓冲负责保证生产者和消费者并发访问时的数据一致性；最新帧策略负责消除历史图像积压；帧序号和等待队列负责实现可靠同步；

一帧背压用于控制后台线程的资源占用；JPEG 完整性校验则负责在进入硬件解码器之前拦截损坏图像。该方案使视觉线程能够获得尽可能新且完整的摄像头图像，并为后续 JPU 解码、VPSS 预处理和 TPU 推理提供稳定的数据输入。

5.3 JPU 硬件解码优化

摄像头输出的 MJPEG 码流由一系列相互独立的 JPEG 图像组成。早期处理链路如果 CPU 完成 JPEG 解码，就需要在单核 C906 上执行熵解码、反量化、逆离散余弦变换（Inverse Discrete Cosine Transform，IDCT）和颜色分量重建等计算密集型操作。这些任务不仅占用大量处理器时间，还会与设备驱动、检测后处理和机器人控制状态机竞争唯一的大核资源。为降低 CPU 负载，系统将 JPEG 解码迁移到 SG2002 内置的 JPEG 处理单元（JPEG Processing Unit，JPU）。驱动通过 `decode_planar_into()` 将经过完整性校验的 JPEG 码流提交给 JPU，由专用硬件完成码流解析和像素重建。CPU 仅负责配置寄存器、提交任务、等待完成中断、检查硬件状态以及解析输出格式，不再逐像素执行 JPEG 解码算法。实测 JPU 解码阶段的平均时延约为 1.6～1.7 ms，说明 JPEG 解码已经从主要的软件计算负担转变为时延较低的固定功能硬件任务。



JPU 的输出格式并不能由应用程序任意指定，而是首先取决于输入 JPEG 的色度采样方式。JPEG 图像的帧起始标记（Start of Frame，SOF）记录了各颜色分量的水平和垂直采样因子。当前摄像头输出的 JPEG 中，亮度分量 Y 的采样因子为 `0x21`，表示水平方向采用二比一的亮度—色度采样，而垂直方向不进行色度下采样，因此实际格式为 4:2:2。JPU 按照这一采样关系自然输出三平面的 YUV422P 数据：640×480 图像的 Y 平面占 307,200 字节，Cb 和 Cr 平面各占 153,600 字节，总数据量为 614,400 字节。NV12 则属于 4:2:0 半平面格式，要求色度在水平和垂直方向都减半，并将 U、V 分量交织存储。若把当前 JPU 输出的 4:2:2 三平面数据直接解释为 NV12，将同时造成色度采样比例和内存布局错误。因此，正式链路保留 JPU 的 YUV422P 输出，再由视频处理子系统（Video Processing Subsystem，VPSS）完成正确的色度重采样、颜色空间转换和模型输入格式生成。对于非 4:2:0 JPEG，驱动中的 `CVI_CAMERA_IOCTL_GET_LATEST_NV12_FRAME` 会拒绝执行不符合输入采样格式的转换。

为了减少解码后大尺寸图像的复制，系统使 JPU 直接将 YUV422P 输出写入 Linux ION 内存分配器提供的物理连续缓冲区。用户态通过 `CameraIonFrameRequest` 向内核传入目标 ION 文件描述符、缓冲区偏移、可用容量、上一帧序号以及请求超时时间。内核函数 `latest_yuv_to_ion()` 根据文件描述符查找对应的 ION 分配对象，检查 `buffer_offset + capacity` 是否超过分配范围，并计算 JPU 可以访问的总线物理地址。完成参数校验后，JPU 以该物理地址作为解码输出基址，依次写入 Y、Cb 和 Cr 三个平面。解码成功后，驱动向用户态返回各平面的偏移量、长度和步长，

以及图像宽高、像素格式、帧序号、采集时间戳和 `jpu_decode_us` 等信息。这样，用户态和 VPSS 可以直接根据驱动返回的元数据访问各个平面，不需要重新推导地址或猜测输出布局。

JPU 运行期间，目标 ION 缓冲区的访问所有权属于硬件设备。用户态在 ioctl 返回之前不得读取或修改相应区域，驱动也只有在 JPU 完成解码并执行必要的内存同步后，才将缓冲区重新交还给用户态和后续 VPSS 使用。这种所有权转换是保证 DMA 数据一致性的必要条件，而不只是应用层约定。如果 CPU 与 JPU 同时访问同一目标缓冲区，即使双方使用的物理地址完全正确，仍可能出现图像撕裂、部分平面尚未写完或缓存内容与物理内存不一致等问题。当前链路通过同步 ioctl 语义明确了设备写入和用户态读取的先后关系，从而保证 VPSS 接收到的是一幅完整、稳定的 YUV422P 图像。

需要说明的是，当前链路并非严格意义上的端到端零复制。`latest_yuv_to_ion()` 在取得最新 JPEG 后，会将几十 KB 的压缩码流复制到临时 `Vec<u8>` 中，然后释放 `LatestFrameStore` 的状态锁，再取得 JPU 状态锁并提交硬件解码。这次压缩数据复制是为了缩短最新帧存储锁的持有时间，同时避免摄像头状态锁和 JPU 状态锁以不同顺序嵌套而形成 ABBA 死锁。如果在整个 JPU 解码期间持续持有最新帧锁，后台 UVC 线程将无法发布下一帧，异步采集与推理重叠也会失去意义。当前实现真正消除的是大尺寸未压缩图像复制：614,400 字节的 YUV422P 图像由 JPU 直接写入源 ION 缓冲区，VPSS 直接读取该物理缓冲区；VPSS 生成的 RGB planar 图像也直接写入目标 ION 缓冲区，再由 TPU 使用。因此，更准确的技术表述是“大图像数据通路零拷贝或低拷贝”，而不是整个处理链路完全不存在数据复制。

总体而言，该项优化并不是简单地用硬件替换软件解码，而是围绕 JPU 建立了一条包含输入校验、物理缓冲区输出、DMA 所有权管理、ABI 校验、位流边界保护、异常帧丢弃、有界重试和硬件复位的完整解码路径。JPU 只负责按照 JPEG 原始采样格式重建像素，不负责将任意 4:2:2 图像直接转换成 NV12 或模型所需的 RGB 格式；损坏的 JPEG 也只能被识别、丢弃和恢复，无法由驱动补回已经缺失的 USB 数据。后续若继续优化，应优先研究压缩码流缓冲区的安全共享和跨设备缓冲区所有权协议，而不应通过错误标注像素格式来换取表面上的路径缩短。

5.4 ION 共享物理缓冲区

在原始视觉处理链路中，不同硬件模块通常分别维护自己的输入和输出缓冲区。图像从一个模块传递到另一个模块时，需要先复制到普通用户态内存，再复制到下一个设备能够访问的缓冲区。对于 640×480 的 YUV422P 图像，单帧数据量为 614,400 字节；对于 640×640 的 RGB planar 图像，单帧数据量达到 1,228,800 字节。如果 JPU、视频处理子系统（Video Processing Subsystem，VPSS）和张量处理器（Tensor Processing Unit，TPU）之间均通过中央处理器（Central Processing Unit，CPU）复制整幅图像，单帧将产生数 MB 的额外内存读写。这些复制不仅直接占用单核 C906 的执行时间，还会与硬件加速器竞争双倍数据速率同步动态随机存取存储器（Double Data Rate Synchronous Dynamic Random-Access Memory，DDR SDRAM）的带宽，进而间接延长 JPU、VPSS 和 TPU 的硬件执行时间。



为减少大尺寸图像在不同设备之间的重复复制，系统建立了两级 ION 共享物理缓冲区。ION 是内存分配器的专有名称，并非英文首字母缩写。其中，source ION 作为 JPU 的输出缓冲区和 VPSS 的输入缓冲区，destination ION 作为 VPSS 的输出缓冲区和 TPU 的输入缓冲区。各硬件模块通过同一缓冲区的物理地址直接衔接，用户态主要负责传递 ION 文件描述符、缓冲区偏移、有效长度、平面步长、图像格式和时间戳等元数据，不再通过普通 `Vec<u8>`在模块之间复制完整的 YUV 或 RGB 图像。由此，图像数据的传递方式从“CPU 读取并重新写入”转变为“硬件在共享物理内存上按顺序交接”.用户态在 `VpssRgbPipeline::open()`中为 source ION 分配 `SOURCE_CAPACITY=2×1024×1024` 字节的物理缓冲区，并使用 `akars-jpu` 作为分配标签。该容量明显大于当前 640×480 YUV422P 图像所需的 614,400 字节，用于容纳内存页对齐、硬件平面对齐、不同 JPEG 采样格式以及未来可能扩展的输入分辨率，同时为内核的偏移和容量检查提供明确上限。当前图像在 source ION 中的有效布局由 JPU 驱动返回：Y 平面起始于 `source_paddr + y_offset`，分辨率为 640×480、步长为 640 字节；Cb 平面起始于 `source_paddr + cb_offset`，分辨率为 320×480、步长为 320 字节；Cr 平面起始于 `source_paddr + cr_offset`，其尺寸和步长与 Cb 平面相同。各平面的偏移、长度和步长均由摄像头/JPU ioctl 明确返回，而不是由用户程序根据固定公式硬编码。即使驱动以后改变平面对齐方式，用户态也可以继续依据应用二进制接口（Application Binary Interface，ABI）提供的元数据构造 VPSS 任务。destination ION 的容量根据模型输入尺寸计算，即 `output_width × output_height × 3`，并向 4 KiB 内存页边界对齐。对于 640×640 模型输入，有效图像载荷为 1,228,800 字节。VPSS 按照 RGB planar 格式依次写入红、绿、蓝三个颜色平面：R 平面从偏移 0 开始，G 平面从偏移 `640×640` 开始，B 平面从偏移 `2×640×640` 开始，每个平面的行步长均为 640 字节。该内存布局与模型要求的批次、通道、高度、宽度（Number, Channel, Height, Width，NCHW）张量布局一致，因此不再需要 CPU 执行通道交织转平面、通道顺序调整或逐像素重排。VPSS 完成写入后，TPU 运行时可以直接使用该缓冲区作为模型输入数据来源。

ION 缓冲区由 `IonAllocation::allocate()`统一申请。分配过程使用直接存储器访问（Direct Memory Access，DMA）一致性内存堆，同时取得 ION 文件描述符、实际分配容量和设备可访问的物理地址。用户态随后通过 `mmap()`建立共享虚拟地址映射，并设置 `PROT_READ | PROT_WRITE` 和 `MAP_SHARED` 属性，使用户态在确有需要时能够访问同一批物理页。`IonAllocation` 对象析构时负责执行 `munmap()`并关闭文件描述符，从而保证正常退出和异常返回路径均不会遗留映射关系或长期占用物理内存。文件描述符、用户虚拟地址和设备物理地址分别服务于对象管理、CPU 访问和硬件访问，但三者对应的是同一个 ION 分配对象。SG2002 的部分多媒体设备和 TPU 寄存器只能接收 32 位物理地址，因此共享缓冲区必须位于低 4 GiB 物理地址范围内。如果分配器返回超过 `0xFFFF_FFFF` 的物理地址，而驱动在写入设备寄存器时直接截断高 32 位，硬件将访问错误的内存

区域。这类错误可能表现为静默数据损坏、总线访问异常、设备超时或无法稳定复现的推理错误。为此，内核 ION 路径使用 `alloc_coherent_pages_dma32()` 分配低地址物理页，从分配阶段保证缓冲区满足 DMA32 约束。设备提交路径还会检查物理基址与有效长度相加后是否溢出 32 位可寻址空间。该约束不能通过提交时简单地把 64 位地址强制转换为 32 位来替代，而必须保证内存分配器、驱动 ABI 和硬件地址寄存器宽度保持一致。

共享物理缓冲区并不意味着多个处理单元可以在任意时刻同时访问同一片内存。低拷贝链路必须配合明确的 DMA 数据所有权状态机。source ION 初始由用户态管理，提交 JPU 任务后转为 JPU 独占写入状态；JPU 完成解码并产生完成中断后，缓冲区所有权转交给 VPSS，VPSS 在该阶段只读取 Y、Cb 和 Cr 平面；VPSS 完成读取后，source ION 才能重新用于下一幅图像。destination ION 同样从空闲状态开始，提交 VPSS 任务后由 VPSS 独占写入；VPSS 完成后，缓冲区转交给 TPU 或张量直接存储器访问（Tensor Direct Memory Access，TDMA）单元读取；只有 TPU 前向推理结束后，destination ION 才能被下一帧覆盖或重新使用。通过 source ION 和 destination ION 的两级共享，系统消除了 YUV422P 解码图像与 RGB planar 预处理图像在设备之间的 CPU 整图复制。JPU 将 614,400 字节的 YUV422P 结果直接写入 source ION，VPSS 直接读取该缓冲区；VPSS 再将 1,228,800 字节的 RGB planar 结果写入 destination ION，TPU 通过物理地址或运行时导入机制读取该结果。CPU 不再承担大图像搬运，只需设置设备参数、传递缓冲区描述信息并维护所有权状态。这不仅减少了 CPU 执行时间，也降低了 CPU 对 DDR 的额外访问，为 JPU、VPSS 和 TPU 保留了更多有效内存带宽。

总体而言，ION 优化的核心并非单纯申请一块物理连续内存，而是建立统一的跨设备缓冲区表示方式，并围绕该缓冲区实现地址范围校验、DMA32 约束、映射属性管理、ABI 元数据传递和设备所有权同步。它使 JPU、VPSS 和 TPU 能够在同一条物理数据通路上顺序处理图像，避免 CPU 成为大图像传递的中间节点。该机制既是降低单帧视觉时延的重要基础，也是后续实现多缓冲流水线和设备间异步并行的前提。

5.5 扩展 VPSS 驱动，直接接收 JPU 输出的 YUV422P

项目早期的视频处理子系统（Video Processing Subsystem，VPSS）驱动主要围绕 NV12 输入格式设计。原有源帧抽象只包含亮度 Y 平面和交织 UV 平面两个物理地址，适用于 4:2:0 半平面图像。然而，当前摄像头输出的运动联合图像专家组（Motion JPEG，MJPEG）采用 4:2:2 色度采样，经过 JPEG 处理单元（JPEG Processing Unit，JPU）解码后得到的是 Y、Cb、Cr 三个平面相互独立的 YUV422P 图像。二者在平面数量、色度采样比例和内存布局上均不兼容，因而在 JPU 输出与 VPSS 输入之间形成了格式断点。如果继续使用原有 NV12 接口，只能将 YUV422P 错误标记为 NV12、由中央处理器（Central Processing Unit，CPU）执行一次 YUV422P 到 NV12 的全帧转换，或者放弃 VPSS 并继续使用 CPU 完成颜色转换和缩放。第一种方案会产生错误图像，后两种方案则会引入整图复制和软件预处理开销。



为消除这一格式断点，内核 VPSS 驱动新增了 `Yuv422PlanarFrame` 数据结构和 `SourceFrame::Yuv422Planar` 源帧类型，同时使用 `RgbPlanarFrame` 和 `DestinationFrame` 描述模型输入端的三平面 RGB 目标图像。用户态与内核态共同定义 `VpssRunYuv422pRgb` 请求结构，并通过 `CVI_VPSS_IOCTL_RUN_YUV422P_RGB` 提交离线硬件任务。该接口直接表达“JPU 输出的 YUV422P 三平面图像经过 VPSS 转换为 RGB planar 三平面图像”的数据契约，不再依赖驱动根据缓冲区数量或格式编号进行隐式推断。由此，JPU 输出可以直接作为 VPSS 输入，不需要在两者之间插入 CPU 颜色格式转换。

YUV422P 输入需要按照 CV181x/SG2002 官方视频缩放器的格式定义配置。驱动将输入格式设置为 `SCL_FMT_YUV422`，为 Y、Cb 和 Cr 分别配置独立的物理基址，并按照 4:2:2 采样关系设置颜色分量。亮度分量采用 8 位采样，水平方向的色度下采样因子为 2，垂直方向的色度下采样因子为 1。对于 640×480 输入图像，Y 平面的行步长为 640 字节，Cb 和 Cr 平面的行步长均为 320 字节；由于 4:2:2 只在水平方向降低色度分辨率，两个色度平面的高度仍然是 480 行。驱动必须使用 JPU 接口实际返回的 `stride_y` 和 `stride_c`，不能假设三个平面的步长都等于图像宽度。格式扩展的关键并不只是增加第三个物理地址字段，还包括采样因子、平面顺序、步长和颜色空间转换参数的一致配置。如果把垂直色度采样因子错误设置为 2，VPSS 会把 4:2:2 输入按照 4:2:0 读取，只处理一半的色度行，最终产生垂直方向的颜色错位；如果交换 Cb 和 Cr 地址，则会产生明显的红蓝通道反转；如果将色度步长错误设置为 640，硬件又会跨越正确的色度行边界。为了保证图像内容正确，驱动需要同时配置输入格式、三个物理地址、两个步长、水平和垂直采样关系，以及与输入颜色范围匹配的颜色空间转换矩阵。

用户态和内核态之间通过固定的应用二进制接口（Application Binary Interface，ABI）描述一次完整 VPSS 任务。当前 `VpssRunYuv422pRgb` 的 ABI 版本为 1，结构体大小固定为 248 字节。请求中包含源、目标 ION 文件描述符，以及 Y、Cb、Cr 三个输入平面的偏移；同时包含源图像宽高、亮度步长和色度步长。几何变换部分描述源裁剪区域、目标内容区域以及完整输出画布的宽高；输出部分描述 R、G、B 三个平面的偏移和步长；控制部分包含边框颜色、任务超时、输入帧序号和时间戳；返回部分则包含执行状态、中断计数、排队时间、启动时间、完成时间、硬件执行时间、输出序号、输出时间戳，以及 IMG、AXI、SCALER 和输出直接存储器访问（Output Direct Memory Access，ODMA）等硬件模块的状态寄存器快照。将这些字段集中在一次 ioctl 请求中的目的，是让一个离线 VPSS 任务完整描述输入格式、颜色转换、裁剪、缩放、letterbox 填充和输出布局。如果将上述处理拆分为多个驱动接口，就需要多次系统调用、多次硬件任务配置以及额外的中间缓冲区；每个中间阶段还会增加新的设备同步点和缓冲区所有权转换。当前设

计使 VPSS 能够在一次硬件调度中完成模型所需的全部图像预处理，既缩短控制路径，也避免生成用户态不需要的中间图像。

VPSS 属于直接存储器访问（Direct Memory Access，DMA）主设备，因此在写入硬件寄存器之前，驱动必须严格验证请求中的缓冲区和几何参数。内核首先检查 ABI 版本和结构体大小是否匹配，再通过源、目标 ION 文件描述符解析受信任的物理内存对象。随后验证 Y、Cb、Cr 三个源平面以及 R、G、B 三个目标平面的地址范围是否完全位于对应 ION 对象内，检查各平面的偏移、长度和物理地址计算是否发生整数溢出，并确认宽度、高度和步长符合硬件约束。驱动还需要保证源裁剪区域没有超出 640×480 输入图像，目标内容区域没有超出输出画布，RGB 三个输出平面之间不存在非法重叠，并且目标缓冲区容量足以容纳完整输出。这些边界检查虽然位于性能关键路径中，但不能为了减少少量软件开销而省略。错误的物理地址、步长或平面长度可能使 VPSS 越过合法缓冲区，通过 DMA 覆盖内核数据、其他进程内存或其他设备正在使用的物理页面。项目的优化原则是减少整图复制、缩短任务层级和合并硬件处理步骤，而不是绕过内存安全检查。与硬件执行时间相比，这些整数范围和布局校验的耗时很小，却是保证长期稳定运行和驱动安全性的必要条件。

完成参数校验后，VPSS 在一次离线硬件任务中完成模型输入预处理。硬件首先从 source ION 读取 Y、Cb 和 Cr 三个平面，并使用 BT.601 有限范围颜色转换矩阵，将 YCbCr 数据转换到 RGB 颜色域；随后按照目标内容区域执行比例缩放，将转换后的图像放置到完整输出画布中；未被有效图像覆盖的区域使用请求指定的边框颜色填充；最后由 ODMA 按照 R、G、B 三个独立平面的形式，将结果写入 destination ION。整个过程中不需要 CPU 逐像素执行 YUV 到 RGB 转换、双线性缩放、边框填充或通道重排。

总体而言，该项优化完成了 JPU 与 VPSS 之间的原生格式衔接，使 YUV422P 三平面图像能够通过 ION 物理缓冲区直接进入 VPSS，并在一次硬件任务中完成颜色空间转换、letterbox、缩放和 RGB planar 输出。它不仅扩展了一个新的像素格式枚举，还系统解决了三平面地址表达、色度采样配置、步长管理、ABI 定义、DMA 边界校验、输出通道验证、设备生命周期管理和工作模式裁剪等问题。最终形成了“JPU YUV422P—VPSS RGB planar—TPU NCHW 输入”的完整硬件预处理链路，消除了原有格式断点以及 CPU 全帧颜色转换和布局重排开销。

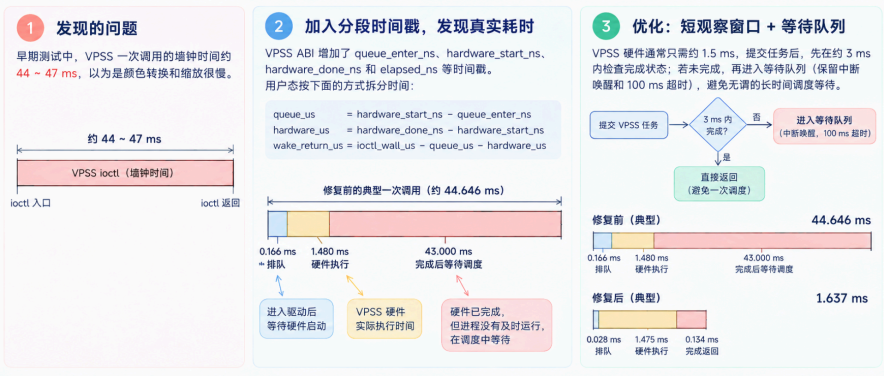
5.6 修复 VPSS 完成后的调度长尾

早期测试中，视频处理子系统（Video Processing Subsystem，VPSS）的单次调用墙钟时间约为 44～47 ms，因此一度被认为是颜色空间转换和缩放效率较低。加入内核分段时间戳后发现，VPSS 硬件从任务启动到完成中断实际只需要约 1.4～1.7 ms，其余数十毫秒主要发生在硬件完成之后：中断处理程序已经确认任务结束，但提交任务的用户线程没有及时恢复运行。如果只测量 ioctl 入口和出口，调度等待就会被错误地统计为 VPSS 硬件执行时间。为区分排队、硬件执行和完成返回三个阶段，VPSS 应用二进制接口（Application Binary Interface，ABI）增加了 `queue_enter_ns`、`hardware_start_ns`、`hardware_done_ns` 和 `elapsed_ns` 等时间戳。用户态通过 `split_vpss_timing()` 计算：

```
queue_us       = hardware_start_ns - queue_enter_ns
hardware_us    = hardware_done_ns  - hardware_start_ns
wake_return_us = ioctl_wall_us - queue_us - hardware_us
```

其中，`queue_us` 表示任务从进入驱动到真正启动硬件的等待时间，`hardware_us` 表示 VPSS 实际执行时间，`wake_return_us` 表示硬件完成后等待任务恢复并返回用户态的时间。如果旧内核不支持详细时间戳，或者时间戳不满足单调递增关系，程序会回退到 `elapsed_ns`，并将该样本标记为不可分段，避免产生负数或失真的统计结果。

该问题的根因是单核 StarryOS 中的中断唤醒与任务调度之间存在较长延迟。VPSS 运行期间，提交线程进入等待队列；硬件完成后，中断请求（Interrupt Request，IRQ）处理程序记录完成状态并唤醒等待者。但是，普通唤醒仅将线程重新放入就绪队列，并不保证其立即获得 CPU。当前任务或其他后台线程仍可能继续使用剩余时间片，导致 VPSS 已经完成，而 ioctl 调用线程仍要等待下一个调度周期。由此可见，硬件完成中断的到达并不等于用户线程已经恢复执行。



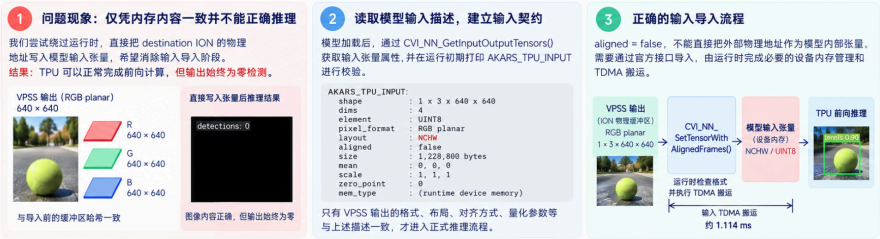
针对正常 VPSS 任务硬件时间约为 1.5 ms 的特点，驱动采用“短观察窗口加等待队列”的两阶段完成机制。任务提交后的最初约 3 ms 内，调用线程对软件完成状态进行有界观察；如果 IRQ 在该窗口内发布完成状态，线程可以直接执行收尾并返回，从而避免一次长时间片级的休眠与重新调度。如果 3 ms 内任务仍未完成，线程就转入等待队列，并保留中断唤醒和 100 ms 绝对超时，防止硬件异常时无限占用 CPU。该机制不是无界忙轮询。3 ms 窗口根据 VPSS 正常执行时间设定，超过窗口后自动进入休眠，因此能够在常规任务中降低返回时延，在异常任务或较长任务中保持系统公平性。观察循环只检查由 IRQ 路径发布的软件完成标志，不直接读取并清除硬件的写一清零（Write One to Clear，W1C）中断状态。中断状态寄存器仍由 IRQ 处理程序统一清理，避免任务线程与中断处理程序竞争清除同一事件而造成完成中断丢失。

修复前，典型 VPSS 墙钟时间约为 44.646 ms，其中硬件执行约 1.480 ms，完成后的调度等待接近 43 ms。修复后，典型墙钟时间降至约 1.637 ms，可分解为约 0.028 ms 排队、1.475 ms 硬件执行和 0.134 ms 完成返回，整体下降约 96.3%。最终正式测试中，StarryOS 的 VPSS 平均时延为 1.643 ms，与 Linux 基线的 1.865 ms 处于同一水平，说明软件调度长尾已基本消除。

该优化表明，设备调用的墙钟时延不等同于硬件执行时延。分析硬件加速器性能时，应分别记录任务入队、硬件启动、硬件完成、中断处理、任务恢复和用户态返回等阶段。3 ms 短观察窗口适用于当前低并发、固定分辨率的专用场景；如果未来引入更高分辨率或多路 VPSS 任务，硬件时间可能超过该窗口，此时功能仍会通过等待队列正常完成，但窗口参数和调度策略需要重新评估，不能通过无限延长轮询时间来追求低延迟。

5.7 模型输入契约校验与正确的 TPU 输入导入

张量处理器（Tensor Processing Unit，TPU）的模型输入不能被简单理解为一段长度相同的字节数据。输入张量的形状、元素类型、颜色通道顺序、内存布局、量化参数、行步长、对齐方式和设备内存类型共同构成模型输入契约。即使缓冲区长度同为 `3×640×640` 字节，如果将蓝绿红（Blue, Green, Red，BGR）误作红绿蓝（Red, Green, Blue，RGB），将高度—宽度—通道（Height–Width–Channel，HWC）误作通道—高度—宽度（Channel–Height–Width，CHW），或者忽略运行时要求的对齐步长，TPU 仍可能正常完成前向计算，但输出结果会稳定地偏离正确值。因此，模型输入格式需要像硬件寄存器接口一样被严格校验。



模型加载后，`YoloModel::open()`依次调用`CVI_NN_RegisterModel()`和`CVI_NN_GetInputOutputTensors()`，获取模型的输入、输出张量描述。程序随后输出`AKARS_TPU_INPUT`诊断信息，包括张量形状、维数、元素格式、元素数量、内存大小、物理地址、内存类型、像素格式、对齐标志、量化比例、零点、均值和缩放系数。只有这些属性与视频处理子系统（Video Processing Subsystem，VPSS）的输出布局一致时，程序才允许进入正式推理流程。该检查可以在运行初期发现模型版本、用户态程序和 VPSS 输出格式之间的不匹配，避免把输入契约错误误判为模型精度问题。

当前正式 640×640 模型的输入契约为：

```
shape        = 1×3×640×640
element      = UINT8
pixel format = RGB planar
layout       = NCHW
aligned      = false
mean         = 0, 0, 0
scale        = 1, 1, 1
zero point   = 0
```

该模型直接接收 8 位无符号整数（Unsigned 8-bit Integer，UINT8）格式的 RGB 三平面数据，不要求 CPU 执行浮点归一化、均值扣除或额外的量化变换。VPSS 因此可以直接生成 `1×3×640×640` 的批次、通道、高度、宽度（Number, Channel, Height, Width，NCHW）输入。但是，`aligned=false` 意味着不能仅凭缓冲区大小一致，就假设运行时允许将任意外部物理地址直接作为模型内部输入张量。正式路径使用`CVI_NN_SetTensorWithAlignedFrames()`，由官方运行时按照模型描述导入 VPSS 输出，并通过张量直接存储器访问（Tensor Direct Memory Access，TDMA）完成必要的数据搬运和内部布局准备。

5.8 TPU ioctl ABI、ION 地址与完成语义修复

CVITEK 神经网络运行时 `libcviruntime.so` 并不直接控制全部张量处理器（Tensor Processing Unit，TPU）寄存器，而是通过`/dev/cvi-tpu0` 的输入输出控制（Input/Output Control，ioctl）接口提交命令缓冲区、等待直接存储器访问缓冲区（DMA Buffer，dma-buf）任务完成并查询执行状态。因此，StarryOS 驱动必须准确兼容 Linux/CVITEK 接口的参数结构、字段大小、输入输出方向、文件描述符语义和返回值规则。此类应用二进制接口（Application Binary Interface，ABI）错误通常不会造成编译失败，而可能表现为运行时重复等待、进入兼容路径、产生额外调度开销，甚至出现偶发超时或静默推理错误。

TPU 驱动不再将用户态传入的整数直接解释为物理地址，而是将其作为 ION 或 dma-buf 文件描述符，从内核维护的缓冲区对象中解析真实物理基址。驱动随后检查命令区域和数据区域是否落在合法分配范围内，验证偏移、长度和地址运算不存在越界或整数溢出，并确认最终设备地址位于 TPU 能够访问的低 4 GiB 物理地址范围。用户态参数则通过受控的虚拟内存指针封装进行读取和写回，避免内核直接解引用不可信用户地址。

该修改统一了视频处理子系统（Video Processing Subsystem，VPSS）与 TPU 的内存来源。VPSS 输出的 destination ION 可以继续作为 TPU 输入导入的基础物理对象，不需要先复制到另一个普通用户态缓冲区或独立的 TPU 专用缓冲区。官方运行时仍可根据模型要求执行张量直接存储器访问（Tensor Direct Memory Access，TDMA），但 VPSS 和 TPU 所引用的是同一套受内核管理的 ION 分配对象，从而维持跨设备共享内存链路。兼容性修复中最关键的问题之一是 `CVITPU_WAIT_DMABUF` 的返回语义。ioctl 系统调用的返回值应表示驱动是否成功解析请求并完成等待操作，而 TPU 硬件任务的执行结果应写入用户提供的 `wait_arg.ret` 字段。如果驱动直接将硬件状态作为 ioctl 返回值，官方运行时可能把已完成的任务误判为系统调用失败，进而重复进入等待或错误恢复流程。修复后，只要参数解析和等待过程本身正常完成，ioctl 就返回 0；任务成功、设备错误或超时状态则通过 `wait_arg.ret` 返回。该语义与官方运行时保持一致，消除了由 ABI 误解引起的重复等待。为保证提交任务与完成事件正确对应，驱动使用提交线程标识和单调递增的任务序号建立任务身份，并将已完成任务写入 `DONE_LIST`。等待方只能取得属于自身请求的完成项，防止多个提交者或运行时内部并行操作错误消费彼此的完成事件。虽然当前机器人视觉程序主要以串行方式执行 TPU 推理，但正确的任务关联仍是驱动 ABI 完整性和未来支持并发任务的基础。

TPU 完成中断请求（Interrupt Request，IRQ）可能同时携带正常完成、TDMA 错误或张量指令单元（Tensor Instruction Unit，TIU）错误状态。驱动不能因为收到完成中断就统一报告成功，也不能在清除中断寄存器后丢失原始错误位。当前中断路径会先读取并保存相关状态寄存器，再发布任务完成状态并唤醒等待者，最后按照硬件要求清理中断。这样，设备错误能够随完成项返回给用户态运行时，而不会被普通完成状态覆盖。当前正式路径启用了 `TPU_DIRECT_EXECUTION=true`，采用同步直通执行方式，使提交线程直接完成命令提交、硬件等待和结果返回，减少提交线程、后台 worker 和完成等待线程之间的调度交接。旧的异步 worker 路径仍被保留，作为 A/B 性能比较、兼容性验证和未来队列化执行的回退方案。两条路径共用相同的用户态 ABI，因此可以在不修改 `libcviruntime.so` 和应用程序的情况下切换执行架构。

总体而言，该项修复并非单纯调整某个 ioctl 返回值，而是统一了用户态运行时、TPU 驱动和 ION 内存管理之间的接口语义。通过规范文件描述符解析、物理地址验证、任务身份管理、完成结果返回和错误状态保存，StarryOS 的 TPU 驱动能够按照官方运行时预期完成命令提交与等待，也为后续消除 worker 调度开销和优化 IRQ 完成路径提供了可靠基础。载需要队列化，可重新启用 worker，而不必重写运行时接口。

5.9 TPU 任务分类与四段时间戳计时机制

如果仅记录 `CVI_NN_Forward()` 的总墙钟时间无法准确定位张量处理器（Tensor Processing Unit，TPU）性能问题。一次调用可能同时包含输入张量搬运、任务排队、命令寄存器配置、张量直接存储器访问（Tensor Direct Memory Access，TDMA）执行、张量指令单元（Tensor Instruction Unit，TIU）计算、中断响应、等待线程恢复和用户态返回等多个阶段。因此，即使测得一次调用耗时 45 ms，也无法仅凭总时间判断瓶颈来自模型计算、内存访问还是操作系统调度。为提高统计的可解释性，驱动根据命令缓冲区头部的描述符数量对任务进行分类。当 `bd_desc_count=0` 且 `tdma_desc_count>0` 时，将任务识别为 `tdma_only`，主要对应模型输入数据导入；当 `bd_desc_count>0` 时，将任务识别为 `compute`，表示任务包含 TIU 计算及相关内部数据搬运；其余无法可靠识别的任务归入 `unknown`。通过这种分类，短时延的输入 TDMA 任务不会与长时延的模型计算任务混合统计。

每个 TPU 任务至少记录四个核心时间区间：

```
submit_to_run  = 驱动收到提交请求到任务实际开始执行
fire_to_irq    = 最终启动寄存器写入到硬件完成IRQ到达
irq_to_resume  = IRQ发布完成状态到等待线程恢复运行
done_to_user   = 内核完成收尾到运行时调用返回用户态
```

此外，驱动使用 `other` 记录寄存器准备、状态读取、任务关联和统计更新等无法归入上述区间的少量开销。`fire_to_irq` 的起点经过专门校准，设置在最终 `TDMA_CTRL` 启动寄存器写入之后，而不是较早的描述符地址配置、时钟控制或中断屏蔽操作。这样，该指标才能准确表示 TPU 命令流从真正启动到产生完成中断的硬件执行区间。内核通过 `TASK_TIMING` 分别输出 `all`、`tdma_only` 和 `compute` 三类任务的调用次数、累计时间、平均时间和最大时间。用户态则把 `CVI_NN_SetTensorWithAlignedFrames()` 的墙钟时间记为“TPU 输入 TDMA 搬运”，把 `CVI_NN_Forward()` 记为“TPU 推理”。结合用户态应用程序接口边界和内核任务时间线，可以判断额外时延产生在运行时、系统调用边界还是驱动内部。例如，若内核 TDMA-only 任务约为 1 ms，而用户态记录为 3 ms，则其余约 2 ms 主要位于运行时封装和系统调用前后；如果 `fire_to_irq` 只有 0.8 ms，但内核任务总时间达到 45 ms，则瓶颈位于排队或任务恢复，而不是 TPU 硬件或 DDR 带宽。

为避免性能观测本身影响测试结果，分段计时使用单调高分辨率时钟，每个任务只保存固定数量的整数时间戳，不在热路径中逐条打印描述符信息。逐帧 TPU 调试输出可以通过运行参数关闭，但累计器仍会在任务结束后输出平均值、最大值和分位数。这样既保留了驱动可观测性，也避免串口日志产生的额外开销被错误计入后处理或其他视觉时延。

当前正式模型的 compute 任务内核平均时间约为 40.387 ms，其中 `submit_to_run` 约 0.002 ms、`fire_to_irq` 约 39.361 ms、`irq_to_resume` 约 0.874 ms、`done_to_user` 约 0.131 ms，其余开销约 0.018 ms。`fire_to_irq` 约占内核 compute 时间的 97.46%，说明早期 worker 排队和任务恢复造成的软件长尾已经基本消除。当前 39～41 ms 的 TPU 推理时间主要来自模型命令流内部的 TIU 计算、TDMA 搬运和存储系统访问，而不是驱动排队或调度延迟。

5.10 TPU 执行链路重构与调度开销消除

早期 TPU 驱动采用异步 worker 架构。用户线程提交任务后先唤醒 TPU worker，由 worker 配置硬件并等待完成中断；中断请求（Interrupt Request, IRQ）到达后再次唤醒 worker，worker 完成状态处理后再通知原始提交线程。一次串行任务因此包含两次额外的线程调度交接。

阶段	异步 worker 路径	同步直通路径
任务提交	用户线程写入队列	用户线程直接取得 `TPU_RUN_LOCK`
硬件配置	唤醒 worker 后由 worker 执行	原提交线程直接执行
硬件等待	worker 阻塞等待 IRQ	原提交线程阻塞等待 IRQ
完成处理	IRQ 唤醒 worker	IRQ 唤醒原提交线程
返回用户态	worker 再通知提交线程	原提交线程直接返回
额外调度交接	至少两次	无中间 worker 交接

在多核系统中，worker 可以与提交线程并行运行；但在单核 C906 上，每次任务交接都可能错过当前调度时间片。早期测试中，`submit_to_worker` 约为 20.899 ms，硬件执行区间约为 20.302 ms，完成后返回用户态约为 5.229 ms。虽然该阶段的硬件统计仍混合了输入 TDMA 与 compute 任务，但已经能够确认软件排队与硬件执行处于同一量级。



当前正式路径启用同步直通执行，由原始提交线程在内核中完成完整任务生命周期：

```
取得TPU_RUN_LOCK
        ↓
解析命令与ION地址
        ↓
配置时钟、复位和设备寄存器
        ↓
写入最终启动寄存器
        ↓
提交线程阻塞等待IRQ
        ↓
IRQ发布完成状态并唤醒提交线程
        ↓
检查状态、生成完成项并返回用户态
```

提交线程首先取得可睡眠的全局 TPU_RUN_LOCK，确保同一时刻只有一个命令流占用 TPU。随后完成任务分类、地址验证、寄存器配置和硬件启动。硬件执行期间，提交线程在 IRQ 等待点休眠并让出 CPU；IRQ 到达后直接唤醒同一个提交线程，由其完成状态检查并返回用户态。任务上下文不再转交给中间 worker。



TPU_RUN_LOCK 必须使用可睡眠锁，不能在持有自旋锁时等待硬件。如果跨越硬件执行阶段持有自旋锁，可能长期禁止调度，并导致单核系统死锁。同步直通仍然采用阻塞式 IRQ 等待，而不是持续忙轮询；其优化对象是任务所有权交接和调度层级。

静态 A/B 测试结果如下：

性能指标	worker 路径	同步直通路径	优化效果
输入阶段	52.830 ms	7.740 ms	降低 85.35%
submit_to_run	22.873 ms	0.002 ms	基本消除
done_to_user	2.958 ms	0.116 ms	降低 96.08%
视觉总时延	100.207 ms	72.042 ms	降低 28.11%
处理帧率	9.96 FPS	13.86 FPS	提高 39.16%

TPU 时钟、模型结构和硬件命令流在两组测试中没有改变，因此性能提升并非来自硬件频率增加，而是来自 worker 排队及线程交接的消除。驱动仍保留 TPU_DIRECT_EXECUTION 开关和原异步 worker 实现，正式版本将其设置为 true。保留旧路径便于进行 A/B 性能对照，也为未来多进程共享 TPU 或批量任务调度提供兼容回退，而不需要修改现有用户态应用二进制接口（Application Binary Interface，ABI）。

同步直通适用于当前单进程、单模型、单帧串行的机器人视觉负载。长时间 compute 任务持有 TPU_RUN_LOCK 时，其他提交者必须等待，因此该方案不适合直接替代通用多任务调度器。如果未来需要同时运行视觉、语音或其他 TPU 模型，应重新引入公平队列、任务优先级或时间片机制。当前设计属于针对确定性工作负载的执行路径裁剪，主要目标是降低单帧响应时延。

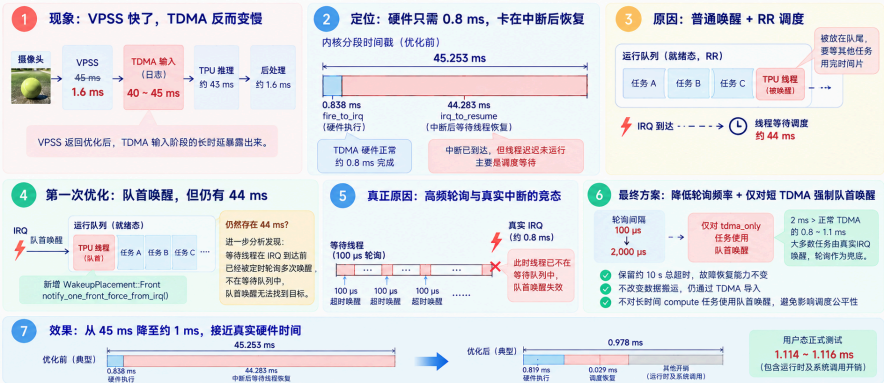
5.11 面向短时 TDMA 的中断唤醒与竞态管理

VPSS 完成返回路径优化后，其平均时延由约 45 ms 下降至约 1.6 ms，但端到端日志中的张量直接存储器访问（Tensor Direct Memory Access，TDMA）输入阶段一度上升至 40～45 ms。这并不表示 VPSS 将计算转移给了 TDMA，而是前级调度瓶颈消除后，后一级原有的中断唤醒长尾变得可见。

内核分段时间戳明确显示，问题不在 TDMA 硬件本身：

TDMA 阶段	优化前典型时间	含义
`fire_to_irq`	0.838 ms	TDMA 硬件执行正常
`irq_to_resume`	44.283 ms	中断后等待线程未及时恢复
内核任务总时间	45.253 ms	主要由调度等待构成

StarryOS 常规 `WaitQueue::notify_one()` 按照普通策略将等待任务加入就绪队列。在时间片轮转（Round-Robin，RR）调度下，刚被中断唤醒的 TPU 提交线程可能被放置在运行队列尾部，需要等待其他任务用完时间片。对于硬件执行时间不足 1 ms 的 TDMA 任务，几十毫秒的恢复等待会完全掩盖硬件性能。



为缩短中断后的恢复路径，调度器新增 `WakeupPlacement::{Normal, Front}`，运行队列支持将被唤醒任务插入队首，等待队列则增加 `notify_one_front_force_from_irq()`。当 TPU 中断处理程序确认当前任务属于 `tdma_only` 时，会将对应提交线程放置到就绪队列前部，使其在中断返回后的下一次调度中优先恢复。第一次引入队首唤醒后，约 44 ms 的长尾仍然存在。进一步分析发现，驱动原先每 100 µs 执行一次超时轮询，而正常 TDMA 约在 0.8 ms 后完成。在真实中断到达之前，等待线程已经多次因定时器超时离开 IRQ 等待队列并重新检查状态。当 IRQ 真正到达时，等待队列中可能已不存在对应线程，队首唤醒因而无法找到提升对象。该问题本质上是高频定时轮询与真实硬件中断之间的竞态。为恢复 IRQ 作为正常完成主路径，驱动将 `WAIT_POLL_INTERVAL_US` 从 100 µs 调整为 2,000 µs，同时保持约 10 s 的总超时。2 ms 大于正常 TDMA 的 0.8～1.1 ms，因此大多数任务会先由真实 IRQ 唤醒；只有在中断丢失或设备异常时，定时轮询才作为兜底机制检查硬件状态。轮询总次数根据总超时重新计算，因此故障恢复窗口没有缩短。

优化前后的典型时间对比如下：

TDMA 阶段	优化前	优化后	结果
`fire_to_irq`	0.838 ms	0.819 ms	硬件时间基本不变
`irq_to_resume`	44.283 ms	0.029 ms	调度长尾基本消除
内核任务总时间	45.253 ms	0.978 ms	恢复到真实硬件量级
用户态正式测试	约 45 ms	1.114～1.116 ms	包含运行时及系统调用开销

因此，“消除 TDMA 时延”并不是删除统计项，也不是取消数据搬运。输入数据仍需通过 TDMA 导入模型要求的存储布局，约 0.8～1.1 ms 的真实硬件时间依然存在。被消除的是 IRQ 到达后约 44 ms 的不必要调度等待。强制队首唤醒只用于 `tdma_only` 短任务。此类任务的硬件时间不足 1 ms，对一个完整调度时间片极为敏感；compute 任务本身约需 39 ms，如果所有 TPU 完成中断都无条件插入队首，可能破坏系统调度公平性。当前驱动通过 `ACTIVE_TDMA_ONLY` 识别短输入任务，对其采用强制前插，而长时间 compute 任务保留较保守的普通唤醒策略。

该优化只改变任务在运行队列中的恢复位置，不改变命令内容、缓冲区数据或硬件完成状态。为避免高频中断造成其他线程饥饿，队首唤醒被严格限制在短 TDMA 任务；2 ms 轮询负责处理中断丢失，10 s 总超时则防止设备异常导致永久阻塞。由此，驱动在保持故障恢复能力的同时，将 TDMA 输入路径恢复到接近真实硬件执行时间的水平。。

5.12 TPU 驱动优化

TPU 性能不仅取决于模型和标称频率，还受到时钟门控、时钟源选择、分频配置、复位顺序、中断状态清理以及任务生命周期管理的共同影响。日志中显示“700 MHz”并不能单独证明 TPU 实际运行在目标频率，也不能说明片上互连（Fabric，FAB）和存储系统处于匹配状态。为此，项目参考 SOPHGO/CVITEK 官方 SG2002/CV181x 驱动及本地 `ref/TPU_ref` 资料，引入 `Cv181xTpuSoc` 管理层，将芯片级时钟与复位控制从通用 TPU 命令执行逻辑中分离。

SoC 初始化按照官方顺序配置 TPU 与 FAB 时钟门控，并管理 TDMA、TPU 和 TPUSYS 复位信号。相关复位采用低有效语义：驱动先清除控制位以断言复位，再置位解除复位，避免在下级模块仍处于活动状态时直接复位上层系统。共享时钟寄存器采用带中断保护的自旋锁执行读—改—写操作，防止不同执行路径并发修改寄存器时覆盖其他位。驱动通过读取 TPLL、FPLL、MIPIMPLL 和 A0PLL 等锁相环（Phase-Locked Loop，PLL）状态，结合时钟多路选择器、旁路状态和从 1 开始计数的分频系数，计算 TPU 与 FAB 的实际工作频率。当前实机结果如下：

时钟或状态	实测值
TPU 频率	700 MHz
FAB 频率	900 MHz
TPLL 频率	1,400 MHz
FPLL 频率	1,500 MHz
MIPIMPLL 频率	900 MHz
TPU 分频寄存器	`0x00020009`
是否重新应用性能分频	`false`

`perf_div_applied=false` 表明启动固件已经将共享 TPLL 和 TPU 分频配置到目标状态，驱动识别到现有配置正确后没有重复改写。共享 PLL 和电压域可能同时服务其他外设，因此不能仅为提高 TPU 频率而擅自修改，否则可能破坏其他设备的稳定性。

TPU 任务按照 `prepare → run → finish` 三个阶段管理：

生命周期阶段	主要职责
`prepare`	打开必要时钟、解除复位、清理遗留状态并确认设备可用
`run`	写入描述符地址，配置动态时钟管理和中断，最终触发命令流
`finish`	保存完成及错误状态，清理中断并完成设备收尾

驱动将解析得到的实际 `tpu_clk_rate` 传递给执行层，用于等待和超时计算，避免继续使用未经硬件验证的固定频率常量。发生超时或错误后，设备也可以依据同一生命周期重新清理状态并恢复运行。

为准确测量硬件命令流，`fire_to_irq` 的起点被移动到最终 `TDMA_CTRL` 启动寄存器写入时刻，终点则由真实完成中断记录。早期计时点位于描述符地址或其他准备寄存器写入之前，会把软件配

置时间混入硬件执行区间。修正后，`fire_to_irq` 能够较准确地表示从硬件命令流启动到完成中断产生的实际时间。

当前正式模型的 compute 任务平均 `fire_to_irq` 约为 39.361 ms，Linux 基线中同一模型的 TPU 推理约为 39.721 ms，两者处于相同量级：

指标	StarryOS	Linux 基线
TPU/FAB 工作频率	700/900 MHz	相同硬件配置
核心硬件执行时间	39.361 ms	39.721 ms
差值	-	约 0.360 ms

该结果说明 StarryOS 不存在明显的 TPU 降频问题，TPU 硬件算力已经与 Linux 基线基本一致。用户态最终观察到的约 1～2 ms 差异更可能来自中断恢复、运行时封装、系统调用边界和测试波动，而不是时钟频率配置错误。

补齐官方时钟、复位和任务生命周期没有带来数量级的时延下降，是因为启动固件原本已经将 TPU 配置为 700 MHz、FAB 配置为 900 MHz。该项工作的主要价值在于排除时钟和复位配置疑点，保证设备异常后能够可靠恢复，使 StarryOS 与官方 Linux 驱动具备可比较的寄存器生命周期，并为硬件时间测量建立可信边界。

目前 compute 任务 97% 以上的时间位于 `fire_to_irq` 区间。该阶段包含模型内部 TIU 计算、权重和特征图 TDMA、片上静态随机存取存储器（Static Random-Access Memory，SRAM）与 DDR 访问以及算子间同步。这部分属于模型命令流的真实硬件成本，无法继续通过提高线程优先级、调整运行队列位置或缩短系统调用消除。若需要进一步降低约 39 ms 的计算时间，应转向模型裁剪、量化、算子融合、编译器调度、片上内存规划和输入尺寸优化。

5.13 TPU 输出直读与轻量化检测后处理

早期检测后处理平均约为 14 ms，主要开销可能来自输出张量复制、通用格式转换、为全部候选框创建对象、低置信度候选参与排序和非极大值抑制，以及逐帧调试日志。当前模型只包含“网球”一个类别，通用多类别处理流程中存在较多冗余，因此用户态对输出读取和候选框筛选顺序进行了针对性简化。

`get_detections()` 首先检查 TPU 输出张量的数据格式。如果输出已经是 32 位浮点数（32-bit Floating Point，FP32），程序通过 `CVI_NN_TensorPtr()` 取得运行时管理的张量地址，并使用 `slice::from_raw_parts()` 建立只读切片，直接解析 42,000 个浮点元素，不再复制到新的 `Vec<f32>`。只有输出为量化格式、脑浮点数（Brain Floating Point，BF16）或其他非 FP32 格式时，才回退到 `tensor_to_f32()` 执行数据转换。

正式模型的输出形状为 `[1,5,8400,1]`，即 5 个通道和 8,400 个候选位置。各通道含义如下：

通道	数据含义
0	检测框中心横坐标 `cx`
1	检测框中心纵坐标 `cy`
2	检测框宽度 `w`
3	检测框高度 `h`
4	网球类别置信度

`parse_yolov8_output()`按照通道跨度访问数据，而不是把输出错误地解释为 8,400 组连续五元组。多类别兼容路径会在类别通道中寻找最大分数及类别编号；对于当前单类别模型，该过程简化为一次连续的置信度扫描。

程序采用“置信度阈值优先”的处理顺序，默认阈值为 0.5。候选分数只有超过阈值后，程序才读取坐标、计算角点并创建 `Detection` 对象。由于 8,400 个候选位置中的绝大多数属于背景，该策略能够显著减少动态对象数量、排序规模和后续非极大值抑制（Non-Maximum Suppression，NMS）的比较次数。优化重点不是盲目提高阈值，而是在保持检测标准不变的情况下尽早淘汰无效数据。

通过阈值筛选的候选框按置信度降序排列，NMS 只在相同批次和相同类别之间计算交并比（Intersection over Union，IoU）：

```
intersection =
    max(0, min(r1,r2) − max(l1,l2))
  × max(0, min(b1,b2) − max(t1,t2))

union = area1 + area2 − intersection
IoU   = intersection / union
```

当 IoU 超过默认阈值 0.5 时，程序抑制置信度较低的重复框，最终一次性保留有效结果。实际网球场景通常只有一个主目标，经过置信度过滤后进入 NMS 的候选数量较少，因此当前后处理平均时延已下降到约 1.6 ms。

模型使用 640×640 输入，而摄像头原始图像为 640×480，因此检测框还需要执行 letterbox 坐标逆映射。当前几何关系为：

参数	数值
缩放比例 `scale`	1
有效内容宽度 `new_w`	640
有效内容高度 `new_h`	480
水平填充 `pad_x`	0
垂直填充 `pad_y`	80

`correct_yolo_boxes()`从模型坐标中减去填充偏移，再除以缩放比例，将预测框恢复到摄像头坐标系，最后裁剪到`[0,639]×[0,479]`范围。如果省略该步骤，检测框的垂直坐标将产生 80 像素偏差，进一步影响网球中心位置、面积比例、转向决策和抓取条件判断。

优化前后的主要变化如下：

优化项目	原始方式	当前方式
FP32 输出读取	复制或通用转换	直接建立只读切片
候选框创建	大量候选提前构造	超过阈值后才构造
排序与 NMS	包含大量低分候选	仅处理有效候选
调试输出	可能逐帧打印	默认关闭详细日志
平均后处理时间	约 14 ms	约 1.6 ms

最新测试中，StarryOS 后处理平均为 1.632 ms，Linux 基线为 0.449 ms。两端使用相同的 Rust 解析、阈值筛选、NMS 和坐标恢复逻辑，因此差异更可能来自张量映射缓存属性、CPU 调度、编译环境、候选数量及缓存冷热状态。

5.14 最终总结

本项目针对 SG2002 单核异构视觉平台，对网球机器人推理链路进行了从采集、解码、图像预处理、共享内存、张量导入、神经网络执行到检测后处理的跨层优化。摄像头侧采用后台异步 UVC 组帧、单调帧序号和一帧在途背压，在不形成历史队列的前提下提供最新完整帧；JPU 侧根据 JPEG 4:2:2 采样格式将 MJPEG 直接解码为 YUV422P，并通过三平面 ION 接口把结果写入设备共享物理内存，同时通过 JPEG 完整性校验、更新帧重试、解码器复位和 1,024 字节位流尾部安全区解决损坏帧与页边界预取问题。

在图像预处理阶段，StarryOS VPSS 驱动由原来的 NV12 输入扩展为符合官方寄存器语义的 YUV422P 三平面输入，并在一次离线任务中完成 BT.601 颜色空间转换、等比例缩放、640×640 letterbox 和 RGB planar 输出。JPU 输出的 source ION 与 VPSS 输出的 destination ION 均采用 DMA32 物理连续分配，完整 YUV 和 RGB 图像不再经过 CPU 大块复制。通过排队、硬件启动和硬件完成时间戳发现，早期约 45 ms 的 VPSS 墙钟中真实硬件仅约 1.5 ms，其余为中断后调度等待；采用 3 ms 有界完成观察与 IRQ 兜底后，VPSS 平均时延降至约 1.6 ms。

TPU 路径首先修正 ION 文件描述符解析、`WAIT_DMABUF` 返回语义、任务完成队列和错误状态传递，使官方 CVITEK 运行时与 StarryOS 内核 ABI 对齐。随后按描述符数量区分 TDMA-only 输入任务与 compute 任务，并将任务生命周期拆分为提交排队、硬件命令流、中断恢复和返回用户态四个区间。针对单模型串行负载，提交线程改为持有可睡眠运行锁直接执行 `prepare/run/wait/finish`，删除提交线程与 TPU worker 之间的两次额外调度交接。对于硬件仅约 0.8 ms 的输入 TDMA，调度器增加 IRQ 队首唤醒；同时将 100 μs 定时轮询改为 2 ms，避免定时器在真实 IRQ 到来前移除等待者。最终 TDMA 内核任务约 1 ms，过去约 44 ms 的 IRQ 后等待被实际消除，而非从统计中隐藏。

驱动还依据 SOPHGO 官方实现补齐 CV181x TPU/FAB 时钟门控、低有效复位顺序、PLL/mux/divider 频率解析及 `prepare/run/finish` 生命周期。实机确认 TPU 工作于 700 MHz、FAB 工作于 900 MHz，且固件已经设置正确分频。将 `fire` 时间点校准到最终启动寄存器后，正式 640×640 模型的硬件命令流约 39.4 ms，占内核 compute 墙钟的 97% 以上，说明软件排队已不再是 TPU 主瓶颈，剩余优化必须面向模型结构、编译和存储访问，而不能继续依靠提高线程优先级。

在正式 640×640 模型下，StarryOS 最新单帧平均结果为：摄像头等待 5.489 ms、JPU 1.645 ms、VPSS 1.643 ms、TPU 输入 TDMA 1.114 ms、TPU 推理 41.309 ms、后处理 1.632 ms、其他视觉开销 0.194 ms，总时延 53.024 ms，平均置信度 93.13%。同硬件 Linux 基线总时延为 54.230 ms。StarryOS 依靠专用摄像头/JPU/VPSS 前端取得约 4.173 ms 优势，抵消了 TPU 与后处理约 2.968 ms 的劣势，最终低约 1.206 ms，即本轮约快 2.22%。这一结果说明当前 StarryOS 已在成功视觉帧平均时延上追平 Linux，并验证了“固定工作负载专用数据通路、硬件共享缓冲区、精确 IRQ 调度和模型契约校验”这一优化路线的有效性。

6 机器人用户态程序优化

6.1 基于视觉尺度反馈的二次曲线渐进接近控制

早期机器人在发现网球后主要采用固定速度或近似线性分段速度向前行驶。当网球距离较近时，视觉采集、模型推理、运动指令下发和底盘制动之间存在响应延迟，机器人可能在接近目标后继续高速前冲，导致网球被撞离工作区域。为降低接近阶段的冲击，系统使用网球检测框面积比例估计目标远近，并通过二次函数连续调节前进速度。

设摄像头图像尺寸为 $ \mathrm{WH} $，网球检测框的左上角和右下角分别为 (x_1, y_1) 和 (x_2, y_2)，则目标面积比例定义为：

$$A = \frac{(x_2 - x_1)(y_2 - y_1)}{WH}$$

在摄像头安装位置和网球尺寸固定的条件下，网球越靠近机器人，检测框面积比例 A 通常越大。虽然该值不是经过标定的真实物理距离，但能够作为接近控制中的视觉距离反馈。

设最大追踪速度为 $v_{\max}$，抓取面积阈值为 A_g，则二次减速模型为：

$$v_q(A) = v_{\max}\left(1 - \frac{A}{A_g}\right)^2, \qquad 0 \le A < A_g$$

当 $A \ge A_g$ 时，机器人停止前进并进入抓取确认：

$$v_q(A) = 0, \qquad A \ge A_g$$

早期算法对比采用：

$$v_{\max} = 45, \qquad A_g = 0.55$$

在该组参数下，原线性或分段控制策略与二次曲线控制的工程标定结果如下。表中的二次速度用于展示曲线的减速趋势，数值包含整数化处理。

网球面积比例 A	线性/分段策略（旧）	二次曲线策略（新）	运动状态
0.03	45，全速	40	略有减速
0.10	45，全速	30	明显变慢
0.20	35	17	中距离减速
0.30	18	8	接近目标，低速缓行
0.40	8	3	接近停止状态
0.55	0	0	停车并进入抓取

从表中可以看出，旧策略在网球面积比例由 0.03 增加到 0.10 时仍保持 45 的最高速度，机器人在已经接近目标后才开始明显降速。二次曲线则从网球刚被发现时就逐步降低速度，并在中近距离阶段快速进入低速状态，从而提前抑制底盘动量。

若令归一化接近程度为：

$$t = \frac{A}{A_g}, \qquad 0 \le t < 1$$

则二次速度函数可写为：

$$v_q(t) = v_{\max}(1-t)^2$$

在 $ 0<t<1 $ 范围内：

$$(1-t)^2 < 1-t$$

因此，在最大速度和抓取阈值相同的情况下，二次函数的输出始终低于对应的线性函数。这意味着二次控制会比线性控制更早、更明显地降低车辆速度，尤其适合视觉反馈存在一定周期延迟的机器人接近任务。

考虑到底盘电机存在静摩擦和低速死区，正式程序还增加了最低有效速度 $v_{\min}$。最终控制函数为：

$$v(A) = \begin{cases} \max\left[v_{\min}, \operatorname{round}\left(v_{\max}\left(1-\frac{A}{A_g}\right)^2\right)\right], & 0 \le A < A_g \\ 0, & A \ge A_g \end{cases}$$

当前进一步限速后的参数为：

$$v_{\max} = 27, \qquad v_{\min} = 6, \qquad A_g = 0.55$$

其中，最大追踪速度由 45 降低至 27，降低幅度为：

$$\frac{45-27}{45} \times 100\% = 40\%$$

因此，前述对比表反映的是引入二次曲线时相对于旧控制策略的减速效果；当前程序又在此基础上将最大速度进一步降低 40%。同时，程序将实际输出速度限制为不低于 6，所以表中理论值为 3 的近距离阶段，在当前底盘上会执行速度 6，避免车辆因低于电机死区而停滞。只有面积达到抓取阈值 $A_g = 0.55$ 时，速度才直接变为 0。

纵向接近速度只在网球位于允许的横向中心区域时生效。设网球检测框水平中心为 c_x，夹爪标定目标中心为 c_{target}，中心容差为 Δc，则完整控制决策为：

$$u(A, c_x) = \begin{cases} \text{TurnLeft}, & c_x < c_{\text{target}} - \Delta c \\ \text{TurnRight}, & c_x > c_{\text{target}} + \Delta c \\ \text{Forward}\ (\ v(A)), & \left|c_x - c_{\text{target}}\right| \le \Delta c \end{cases}$$

由此，机器人将横向对准与纵向接近分离：横向使用中心死区和短时差速脉冲完成纠偏，纵向使用检测框面积反馈和二次曲线控制速度。当面积达到 0.55、网球中心进入近球抓取窗口且连续两帧满足条件后，底盘停止并启动机械臂抓取序列。

该优化使机器人形成“远距离较快接近—中距离主动减速—近距离低速缓行—达到阈值停车抓取”的渐进式运动过程，减少了固定高速接近造成的制动过晚、近距离过冲和撞球现象。

主要问题是：`\(...\)`、`\[...\]` 不是通用 Markdown 公式定界符，很多渲染器不认。应改成行内 `$...$`、块级 `$$...$$`。下面这版已改好，可直接复制到支持 KaTeX/MathJax 的 Markdown 中。

6.2 可识别红色容器

当前 YOLOv8 模型只负责识别网球，并未训练红色容器类别。考虑到测试环境相对干净，且场地中只有一个显著红色接球容器，系统在用户态增加了基于 YUV 颜色阈值的轻量级红色目标检测算法，从而避免重新采集数据、标注并训练双类别神经网络。

红色检测直接作用于 JPU 输出的 640×480 YUV422P 图像，不需要先转换为 RGB。设像素位置为 (x, y)，其亮度和色度分量分别为 $Y(x,y)$、$U(x,y)$ 和 $V(x,y)$，则红色像素判定函数为：

$$R(x,y) = \begin{cases} 1, Y(x,y) \geq 24 \wedge U(x,y) \leq 115 \wedge V(x,y) \geq 155 \\ 0, \text{其他情况} \end{cases}$$

其中，亮度下限用于排除暗部噪声，较低的 Cb 分量和较高的 Cr 分量用于描述红色区域。程序遍历图像后统计红色像素数量：

$$N_R = \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} R(x,y)$$

当 $N_R < 200$ 时，当前帧被认为没有有效红色目标；当红色像素数量达到最低要求后，程序进一步计算红色像素质心：

$$c_x = \frac{1}{N_R} \sum R(x,y)x, \qquad c_y = \frac{1}{N_R} \sum R(x,y)y$$

同时计算红色区域外接矩形 $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$。由此得到红色像素覆盖率、外接框宽度覆盖率和高度覆盖率：

$$\rho_R = \frac{N_R}{WH}$$

$$\rho_W = \frac{x_{\max} - x_{\min} + 1}{W}, \qquad \rho_H = \frac{y_{\max} - y_{\min} + 1}{H}$$

其中，ρ_R 用于描述图像中实际红色像素的占比，ρ_W 和 ρ_H 用于描述红色容器在水平方向和垂直方向对视野的覆盖程度。

红色容器只有在机器人已经完成抓球、`holding_ball=true` 时才会成为有效运动目标。空夹状态下，即使摄像头观察到大面积红色区域，用户态状态机也会将其映射为 `Ignore`，防止机器人在尚未抓到网球时错误驶向接球容器。

持球状态下，机器人根据红色区域的水平质心控制方向。当目标位于中心死区左侧时执行左转，位于右侧时执行右转，进入中心区域后向前抵近。由于红色容器在近距离时包围框较宽，程序使用自适应中心死区：

$$\Delta c_R = \min\left(\max\left(70, \frac{w_R}{4}\right), \frac{W}{3}\right)$$

其中，w_R 为红色外接框宽度。该方法能够避免宽大的红色目标已经覆盖画面中心时，仅因质心小幅波动而反复左右转向。转向脉冲限制在 25～60 ms，进一步减少过冲。

机器人向红色容器前进时，根据外接框面积比例采用三档速度：

红色外接框面积比例	前进速度	控制目的
小于 0.45	12	快速接近远距离容器
0.45～0.75	9	中距离减速
大于或等于 0.75	6	近距离低速抵近

最终停车条件为：

$$\text{Stop} = (\rho_R \geq 0.98) \vee (\rho_R \geq 0.55 \wedge \rho_W \geq 1.00 \wedge \rho_H \geq 0.98)$$

即红色像素几乎覆盖全画面，或者红色区域具有足够像素密度且外接框达到画面宽度 100%、高度 98%。停车条件必须连续三帧成立，系统才由 `ApproachRedContainer` 切换到 `ReleaseTennis`，从而避免单帧颜色噪声或检测框抖动触发误放球。

在红车搜索与抵近阶段，用户态调用 `next_yuv()` 直接取得 YUV422P 图像，不执行 VPSS 颜色转换和 TPU 推理。这种按任务状态裁剪视觉链路的方式减少了不必要的硬件处理，同时不会影响网球检测阶段的识别能力。

6.3 摄像头完全遮挡后的脱困与视觉恢复

机器人抵近红色容器后，容器会覆盖接近 100% 的摄像头视野。此时摄像头无法观察远处网球，刚放下的网球和红色容器还可能持续占据画面。如果程序立即恢复普通追球逻辑，可能出现三种异常行为：将刚放下的球重新识别为新目标、把近距离大目标判定为“过近”并反复后退，或者在看不到远处网球时高速旋转，导致下一目标从视野中快速掠过。

该问题不是神经网络失效，而是机器人进入了暂时不可观测状态。由于红色容器完全遮挡摄像头，系统无法仅依靠当前图像判断安全撤离方向。因此，用户态在放球后引入一段确定性的开环脱困动作，先恢复摄像头视野，再重新进入视觉闭环。

完整恢复过程如下：

```
确认车辆停止
    ↓
servo2张开至200°并回读确认放球
    ↓
servo2恢复至98°空夹位置
    ↓
左右轮以(-16,-16)直线后退500 ms
    ↓
制动并停止20 ms
    ↓
左右轮以(6,-6)原地顺时针旋转
    ↓
重新发现网球后恢复正常追踪控制
```

后退阶段左右轮速度相同，因此机器人沿原方向直线离开红色容器；后退结束后先执行制动和停止，再开始两侧轮速相反的原地旋转。将后退和转向划分为两个独立阶段，可以避免二者叠加成一边倒退一边旋转的弧线运动。

为了防止机器人重新识别刚刚放下的网球，状态机在放球后设置 20 个视觉帧的保护窗口。在该窗口内，即使检测器返回网球框，程序也暂时忽略检测结果，继续执行原地搜索：

$$N_{\text{guard}} = 20$$

保护窗口结束后，如果仍未检测到有效网球，机器人继续以速度 $(6, -6)$ 低速顺时针旋转。低速搜索状态由 `slow_search_after_deposit` 保存，不会在下一帧立即恢复到普通搜索速度 12。只有真正检测到新的网球后，该状态才被清除，并恢复正常的追球转向速度和二次曲线接近控制。

放球后阶段	左轮速度	右轮速度	持续条件
直线后退	−16	−16	500 ms
制动稳定	0	0	20 ms
低速顺时针搜索	6	−6	直到重新发现网球
正常追踪	动态控制	动态控制	目标有效时

机械臂动作执行期间，后台 UVC 任务仍在采集图像。用户态结束阻塞式机械臂动作后，不会继续处理动作开始前的历史帧，而是按照帧序号请求最新图像。因此，机器人完成后退和转向后能够基于较新的环境状态恢复视觉控制，避免旧帧积压进一步延迟重新定位。

这套机制将“完全遮挡时的开环脱困”和“视野恢复后的闭环追踪”结合起来，使机器人能够从红色容器近距离遮挡状态中稳定退出，并继续寻找下一颗网球。

6.4 多目标识别与基于视觉距离的抓取排序

当前 YOLOv8 模型属于单类别网球检测模型，但能够在同一帧中输出多个网球实例。需要区分“多类别识别”和“多实例识别”：模型并未同时学习网球和红色容器两个神经网络类别，而是能够同时检测画面中的多颗网球；红色容器由独立的 YUV 颜色算法识别。两种感知方式由用户态状态机统一调度。

YOLOv8 输出经过置信度阈值过滤和非极大值抑制后，可以得到网球检测集合：

$$\mathcal{D} = \{D_i = (B_i, s_i) \mid i = 1, 2, ..., n\}$$

其中，B_i 为第 i 个网球检测框，s_i 为对应置信度。默认置信度阈值为 0.5，非极大值抑制的交并比阈值为 0.5，用于删除同一网球产生的重叠重复框。

对于每个候选目标，程序计算检测框面积：

$$S_i = w_i h_i$$

并计算其相对整幅图像的面积比例：

$$A_i = \frac{w_i h_i}{W H}$$

在网球实际尺寸近似一致、摄像机参数固定的条件下，距离摄像头较近的网球通常具有更大的检测框。因此，系统采用最大面积优先策略选择当前控制目标：

$$i^* = \arg\max_i S_i$$

$$D^* = D_{i^*}$$

该策略不是严格的三维距离测量，而是利用单目图像中的目标尺度进行距离近似。与单纯选择最高置信度目标相比，最大面积策略更符合抓取任务需求：置信度最高的球不一定距离最近，而优先抓取近处网球通常具有更短的移动距离、更大的图像特征和更稳定的抓取条件。

目标选择流程如下：

```
一帧中检出多个网球
        ↓
置信度阈值过滤
        ↓
NMS消除重复检测框
        ↓
计算每个检测框面积
        ↓
选择面积最大的网球
        ↓
对该目标执行转向、减速和抓取
```

系统在任意时刻只追踪一个主目标，以避免多个网球同时影响转向决策。选定目标后，其水平中心用于横向对准，面积比例用于二次曲线减速和抓取距离判断。机器人完成抓取后切换到红色容器状态，不再处理其他网球；放球并退出红色容器遮挡区域后，系统重新对当前画面中的全部网球执行检测和最大面积选择，从而开始下一轮任务。

完整多目标任务状态可表示为：

多球检测 → 选择最近目标 → 追踪与抓取 → 寻找红色容器 → 放球 → 重新检测剩余网球

用户态还提供 `--max-deposits N` 参数，用于指定需要完成的抓取和放回次数。每次经过机械臂位置回读确认的放球动作完成后，计数器增加 1；当完成数量达到 N 时，程序停止底盘并安全退出。该功能使机器人能够连续完成三球或更多目标的顺序抓取，并为多目标任务提供明确的终止条件。

需要指出，最大检测框面积只能作为距离的近似指标。当网球被遮挡、位于画面边缘或检测框尺寸存在明显误差时，面积排序可能不完全等价于真实距离排序。若后续需要更严格的最近目标选择，可进一步引入摄像头标定、目标直径模型、单目测距或深度传感器。当前方案计算成本低，不需要额外硬件，已经能够满足固定场地中的多网球顺序抓取需求。

7 StarryOS 与 Linux 视觉性能对比分析

在使用正式 640×640 网球模型完成三球任务的实测中，StarryOS 的单帧摄像头等待、JPU 解码和 VPSS 处理合计为 6.087 ms，TPU 输入 TDMA 搬运为 1.116 ms，TPU 前向推理为 41.418 ms，检测后处理为 1.629 ms，其他视觉开销为 0.189 ms，单帧视觉总时延为 50.441 ms，平均检测置信度为 90.83%。与早期依赖 CPU 完成图像转换和缩放的处理方式相比，当前链路将 JPEG 解码、图像预处理和神经网络计算分别交给 JPU、VPSS 和 TPU，并通过 ION 物理缓冲区在硬件模块之间共享图像数据，显著减少了大图复制、CPU 计算和调度等待开销。

7.1 测试结果概述

本次测试在同一 SG2002 硬件平台上，采用相同的 640×640 网球检测模型和基本一致的视觉处理流程，对 StarryOS 与 Linux 基线进行了单帧平均性能对比。两套系统的主要处理链路均可概括为：摄像头采集 MJPEG 图像、JPU 硬件解码、VPSS 硬件预处理、TPU 输入搬运、TPU 前向推理以及 YOLO 检测后处理。

表 StarryOS 和 Linux 基线在相同测试条件下单帧性能对比表

指标	StarryOS	Linux 基线	差值（StarryOS − Linux）	对比 cha ru 结果
摄像头等待	5.489 ms	0.051 ms	+5.438 ms	Linux 更低
JPU 解码	1.645 ms	11.033 ms	−9.388 ms	StarryOS 更低
VPSS	1.643 ms	1.865 ms	−0.222 ms	StarryOS 更低
摄像头/JPU 等待 + VPSS	8.776 ms	12.949 ms	−4.173 ms	StarryOS 更低
TPU 输入 TDMA 搬运	1.114 ms	0.971 ms	+0.143 ms	Linux 更低
TPU 推理	41.309 ms	39.721 ms	+1.588 ms	Linux 更低
后处理	1.632 ms	0.449 ms	+1.183 ms	Linux 更低
其他视觉开销	0.194 ms	0.140 ms	+0.054 ms	Linux 更低
视觉总时延	**53.024 ms**	**54.230 ms**	**-1.206 ms**	**StarryOS 低约 2.22%**
单帧平均置信度	**93.13%**	**93.62%**	**-0.49 个百分点**	**基本接近**

从总量上看，StarryOS 的单帧视觉总时延为 **53.024 ms**，比 Linux 基线的 54.230 ms 减少 1.206 ms，按 Linux 基线计算约快 2.22%。这个结果表明，经过摄像头、JPU、VPSS 和 TPU 驱动优化后，StarryOS 的成功视觉帧处理性能已经与 Linux 处于同一水平，并在本轮测试中略有领先。但是，这一领先并不是所有阶段都更快，而是 StarryOS 在图像解码和预处理阶段取得的优势抵消了其在 TPU 推理、后处理等阶段的额外开销。

7.2 各项指标的含义以及计算方式

视觉总时延的计算公式如下：

```
视觉总时延
  = 摄像头等待
  + JPU 解码
  + VPSS
  + TPU 输入 TDMA
  + TPU 推理
  + 后处理
  + 其他视觉开销
```

系统采用分段插桩方式测量单帧视觉链路。用户态使用单调时钟，在摄像头、VPSS、TPU 运行时和后处理函数调用前后记录时间；StarryOS 内核在 JPU 解码、VPSS 硬件启动与完成中断、TPU 命令触发与中断恢复位置补充纳秒级时间戳。每个成功完成的网球视觉帧生成一组阶段数据，测试结束后计算单帧平均值：

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N} T_i$$

总体探针布局如下：

```
t0：开始处理一帧
  |
  ├─ t1：调用摄像头/JPU ioctl
  |    ├─ k1：内核开始JPU解码
  |    └─ k2：JPU解码完成
  ├─ t2：摄像头/JPU ioctl返回
  |
  ├─ t3：提交VPSS ioctl
  |    ├─ v1：进入VPSS内核接口
  |    ├─ v2：启动VPSS硬件
  |    └─ v3：VPSS完成中断
  ├─ t4：VPSS ioctl返回
  |
  ├─ t5：开始导入TPU输入张量
  ├─ t6：输入TDMA完成
  |
  ├─ t7：调用CVI_NN_Forward()
  ├─ t8：CVI_NN_Forward()返回
  |
  ├─ t9：开始解析输出张量
  ├─ t10：NMS及坐标恢复完成
  |
  └─ t11：本帧视觉处理结束
```

指标	指标含义	计算方式	计时范围	探针位置
摄像头等待	视觉线程请求下一幅可用图像后，等待压缩帧就绪并取得该帧的时间	$T_{\text{camera}} = (t_2 - t_1) - (k_2 - k_1)$	等待新帧、取得最新帧及摄像头接口附加开销	用户态在 `t1`、`t2` 记录摄像头/JPU ioctl 墙钟，内核在 `k1`、`k2` 记录 JPU 解码时间，两者相减得到摄像头等待残差
JPU 解码	将 MJPEG 压缩帧提交给 JPEG 处理单元并取得 YUV422P 解码结果的时间	$T_{\text{JPU}} = k_2 - k_1$	JPU 提交、硬件解码、完成等待及状态检查	StarryOS 内核 `decode_jpeg_into_ion()` 中，`k1` 位于 `decode_planar_into()` 调用前，`k2` 位于解码完成及输出状态检查后
VPSS	完成 YUV422P 到 RGB 的颜色空间转换、缩放、letterbox 填充及 RGB planar 排布的时间	$T_{\text{VPSS}} = t_4 - t_3$；其中 $T_{\text{submit}} = v_2 - v_1$，$T_{\text{hardware}} = v_3 - v_2$，$T_{\text{return}} = (t_4 - t_3) - (v_3 - v_1)$	请求校验与硬件启动、硬件处理、中断唤醒及 ioctl 返回	用户态在 `t3`、`t4` 记录 ioctl 墙钟；内核在 `v1` 记录请求入口、`v2` 记录硬件启动、`v3` 记录完成 IRQ
摄像头/JPU 等待 + VPSS	从请求一帧图像到获得可输入 TPU 的 RGB planar 图像为止的前端视觉时延	$T_{\text{front}} = (t_2 - t_1) + (t_4 - t_3)$	从请求图像到获得模型输入图像	由摄像头/JPU ioctl 墙钟与 VPSS ioctl 墙钟相加，无独立硬件探针
TPU 输入 TDMA 搬运	在模型正式计算前，将 VPSS 输出准备到 TPU 输入张量所需位置的时间	$T_{\text{TDMA}} = t_6 - t_5$	将 VPSS 输出导入 TPU 输入张量	`t5` 位于 `CVI_NN_SetTensorWithAlignedFrames()` 调用前，`t6` 位于该调用返回后
TPU 推理	从提交模型命令流到前向推理完成并返回的时间	$T_{\text{forward}} = t_8 - t_7$	命令提交、模型计算、内部访存、中断等待及运行时返回	`t7` 位于 `CVI_NN_Forward()` 调用前，`t8` 位于该调用返回后；内核另在命令触发、完成 IRQ 和任务恢复处设置诊断探针
后处理	解析 TPU 的 FP32 输出，执行置信度过滤、NMS 和检测框坐标恢复的时间	$T_{\text{post}} = t_{10} - t_9$	输出解析、阈值过滤、NMS 及检测框坐标恢复	`t9` 位于 `get_detections()` 调用前，`t10` 位于 `nms()` 与 `correct_yolo_boxes()` 完成后
其他视觉开销	无法归入上述主要阶段的剩余视觉处理时间	$T_{\text{other}} = (t_{11} - t_0) - [(t_2 - t_1) + (t_4 - t_3) + (t_6 - t_5) + (t_8 - t_7) + (t_{10} - t_9)]$	状态整理、时间戳记录、函数边界及取整残差	由单帧视觉外层墙钟扣除五个已明确计时的主要区间得到，底盘和机械臂动作不计入 `t11 - t0`
视觉处理总时间	从前一帧视觉处理开始处，到检测结果写入共享状态并完成本帧视觉处理为止的墙钟时间	$T_{\text{vision}} = T_{\text{camera}} + T_{\text{JPU}} + T_{\text{VPSS}} + T_{\text{TDMA}} + T_{\text{forward}} + T_{\text{post}} + T_{\text{other}}$	从前一帧视觉处理开始处，到检测结果写入共享状态并完成本帧视觉处理为止，含底盘和机械臂动作	`t0` 位于单帧视觉处理开始处，位于检测结果写入共享状态处的检测框 `t11` 位于本帧视觉处理结束处
视觉帧率	单位时间内完成的视觉处理帧数，用于反映视觉前端的整体吞吐能力	$\text{FPS} = \frac{1}{T_{\text{vision}}}$	以完整视觉处理周期为统计对象	由 `StageTimingSample` 累计若干帧后统计得到
检测框平均置信度	最终保留检测框的置信度均值，用于反映检测结果的整体可靠程度	$\overline{C} = \frac{1}{N_d}\sum_{i=1}^{N_d} s_i$	检测框已经生成并完成阈值过滤与 NMS 之后	在 `ConfidenceTotals::record()` 中累计其置信度

其中，StarryOS 的“摄像头等待”采用摄像头/JPU ioctl 墙钟减去成功 JPU 解码时间得到，因此它是一个残差指标，不等同于纯 USB 传输时间，还可能包含等待新帧、JPEG 临时复制、锁操作和系统调用开销。VPSS 的提交时间表示内核接收请求到硬件启动之间的时间，其中包含参数校验、设备锁等待和寄存器配置，并非单纯的任务排队时间。TPU 输入 TDMA 搬运和 TPU 推理均采用用户态 API 墙钟时间，其中同时包含运行时封装、驱动调用、硬件执行、中断等待和返回用户态的开销。

StarryOS 的 JPU、VPSS 和 TPU 可以通过内核时间戳进一步区分硬件执行与软件调度开销；Linux 基线受厂商接口限制，摄像头采用 `poll() + VIDIOC_DQBUF` 墙钟，JPU 采用 `CVI_VDEC_SendStream() + CVI_VDEC_GetFrame()` 墙钟，VPSS 采用 `CVI_VPSS_SendFrame() + CVI_VPSS_GetChnFrame()` 墙钟。因此，单独的摄像头等待和 JPU 解码存在一定的接口边界差异，“摄像头/JPU 等待 + VPSS”和视觉总时延更适合评价系统级性能。所有均值只统计成功完成完整视觉链路的网球帧.

7.3 表格结果解释

StarryOS 在图像前端取得了最明显的优势。其“摄像头/JPU 等待 + VPSS”为 8.776 ms，比 Linux 的 12.949 ms 少 4.173 ms，降低约 32.23%。进一步分解可见，StarryOS 的摄像头等待比 Linux 多 5.438 ms，但 JPU 解码少 9.388 ms，VPSS 又少 0.222 ms。也就是说，StarryOS 前端的净优势几乎全部来自 JPU 路径；该优势不仅补偿了摄像头等待的劣势，还为后续阶段留下了约 4.173 ms 的时延余量。

进入 TPU 阶段后，Linux 更占优势。StarryOS 的 TDMA 比 Linux 多 0.143 ms，TPU 推理多 1.588 ms，后处理多 1.183 ms，其他视觉开销多 0.054 ms，这四部分合计使 StarryOS 比 Linux 多消耗约 2.968 ms。最终，前端节省的 4.173 ms 抵消了后端多出的约 2.968 ms，形成约 1.205 ms 的净优势，与报告总时延差 1.206 ms 基本一致，剩余 0.001 ms 来自逐项四舍五入。

因此，对本轮数据最准确的描述不是“StarryOS 的所有驱动都比 Linux 快”，而是“StarryOS 依靠更短的专用 JPU 前端路径，在后端仍略慢于 Linux 的情况下实现了略低的单帧总时延”。

7.4 StarryOS 快于 Linux 的原因分析

1)面向固定工作负载的专用内核通路

Linux 基线使用标准 V4L2 摄像头接口以及 CVITEK Multimedia Framework 中的 VDEC、VPSS 等通用组件。其优势是接口完整、兼容性强，但一次处理会经过 `poll()`、`VIDIOC_DQBUF`、VDEC 码流提交、VDEC 输出帧获取、VPSS 送帧和取帧等多层框架接口。

StarryOS 则针对当前固定的 SG2002、UVC MJPEG 4:2:2 摄像头和 640×640 模型构建了专用数据路径。UVC 后台任务负责持续组帧，视觉线程通过专用 ioctl 获取最新帧；JPU 直接将解码结果写入预先分配的 ION 物理缓冲区；VPSS 再直接读取该缓冲区。专用路径减少了通用媒体框架的对象管理、队列切换和多次 API 往返，因此在固定工作负载下可以获得更低的控制路径开销。

2) 最新帧覆盖策略避免队列积压

StarryOS 摄像头驱动采用异步采集和“只保留最新完整帧”的策略。当摄像头产生图像的速度高于推理速度时，新帧可以覆盖尚未消费的旧帧，前台不会沿着历史队列逐帧追赶。这种设计降低了机器人闭环控制中的帧龄，也避免了排队时延持续增长。

Linux V4L2 同样具备异步环形缓冲能力，但其基线程序按照 DQBUF、处理、QBUF 的标准队列语义消费帧。两者的目标并不完全相同：Linux 更强调通用缓冲管理，StarryOS 更强调机器人控制所需的最新状态。因此，StarryOS 的设计更适合“允许丢旧帧、不能积压”的实时控制任务。

3) JPU 解码结果直接进入共享物理缓冲区

StarryOS 的 JPU 输出直接写入同时具有 CPU 虚拟地址和硬件物理地址的 ION 缓冲区，后续 VPSS 使用同一物理缓冲区，不需要把完整的 640×480 YUV 图像复制到普通用户态缓冲区。JPU、VPSS 和 TPU 之间以物理地址和共享内存衔接，是 StarryOS 前端获得低时延的重要基础。

Linux 的 VDEC/JPU 同样使用硬件缓冲区，并非纯 CPU 拷贝实现；但是 Linux 计时范围明确包含 `CVI_VDEC_SendStream()` 和阻塞式 `CVI_VDEC_GetFrame()` 的完整 SDK 墙钟时间。该通用 VDEC 通道还涉及输入码流排队、输出帧管理以及框架同步。StarryOS 的 `jpu_decode_us` 取自更专用的内核解码路径，因而报告值明显更低。

需要强调的是，两端使用的是同一块 SG2002 JPU 硬件。1.645 ms 与 11.033 ms 的巨大差距不应直接解释为 JPU 硬件算力差异，更合理的解释是驱动路径、框架开销、排队方式以及计时边界共同造成了差异。前端合计 8.776 ms 对 12.949 ms，比单独的 JPU 数字更适合支撑系统级结论。

4)VPSS 专用离线任务开销较小

StarryOS 通过 `/dev/cvi-vpss0` 的专用离线任务一次完成 YUV422P 到 RGB、缩放、letterbox 和三平面输出，并通过 IRQ 等待硬件完成。实测 1.643 ms 与 Linux 的 1.865 ms 只相差 0.222 ms，说明两端 VPSS 硬件执行能力基本一致，StarryOS 的任务配置和中断返回路径没有引入明显额外开销。StarryOS 略快可能来自更短的专用 ioctl 路径，也可能处于测量波动范围内，不宜将 0.222 ms 解释为硬件性能差异。

5)内核调度链路已经过针对性简化

StarryOS 早期 TPU 路径曾存在提交线程、worker 和完成等待线程之间的多次调度交接。经过优化后，推理任务改为更直接的串行提交，TPU IRQ 唤醒后的调度位置也得到改善。此前分段计时表明，worker 排队和 IRQ 恢复造成的几十毫秒延迟已经被消除。虽然当前 TPU 墙钟仍略慢于 Linux，但这些改动使 StarryOS 不再因通用调度链路而出现数量级上的劣势，并为本轮总时延追平 Linux 提供了基础。

7.5 StarryOS 慢于 Linux 的可能原因

1) 摄像头等待受到请求相位影响

StarryOS 的摄像头等待为 5.489 ms，而 Linux 只有 0.051 ms。Linux 的计时范围是从调用 `poll()` 到 `VIDIOC_DQBUF` 完成；当程序请求时 V4L2 队列中已经存在完整帧，`poll()` 和 DQBUF 可

以近乎立即返回。该数值更多表示“消费者请求时缓冲区是否已有帧”，而不是摄像头完成一帧传输只需 0.051 ms。

StarryOS 的视觉线程通过带 `last_sequence` 的专用 ioctl 请求序号更新的帧。如果当前最新序号已经消费，调用会等待后台 UVC 任务发布更新帧，因此更容易计入下一帧到达前的剩余时间。这种语义能够避免重复处理同一帧，但会将帧周期相位显式反映在摄像头等待字段中。

因此，5.489 ms 对 0.051 ms 主要体现缓冲语义和计时位置差异，不能据此判断 Linux USB/UVC 驱动的数据传输速度比 StarryOS 快一百倍。若要公平比较摄像头子系统，应进一步统一“USB 帧完成时间戳—消费者取得时间戳”的定义，并同时报告帧率、帧龄、丢帧率和序号跳变。

2) TPU 用户态墙钟仍包含约 2 ms 软件开销

StarryOS 的 TPU 推理为 41.309 ms，比 Linux 多 1.588 ms，约慢 4.00%。内核分段测试已经显示，StarryOS 的 `fire → IRQ` 硬件命令流时间约为 39.4 ms，与 Linux 本轮 39.721 ms 的整个 TPU 推理量级非常接近。这说明 SG2002 TPU、模型命令流和主频并不存在明显的硬件算力差距。

StarryOS 从约 39.4 ms 的硬件完成时间增加到 41.309 ms 的用户态 `CVI_NN_Forward()` 墙钟时间，中间还包含运行时提交、系统调用、IRQ 后任务恢复、驱动收尾以及返回用户态等开销。Linux 的官方驱动采用成熟的 dma-buf、异步 submit/wait、硬件 IRQ 和等待队列机制，其任务生命周期和运行时配合更完整，因而可能在完成通知和返回用户态方面更紧凑。

此外，TPU 命令流会与 DDR、VPSS、USB 等模块共享内存系统。即使 TPU 与 FAB 分别工作在相同的 700 MHz 和 900 MHz，测试瞬间的 DDR 争用、缓存状态和中断负载仍可能造成 1～2 ms 的墙钟波动。因此，在没有同轮 `fire → IRQ`、IRQ 恢复、运行时返回和 DDR 带宽数据之前，不能把全部 1.588 ms 都归因于 TPU 驱动本身。

3)TDMA 差距较小，可能来自 DMA/cache 路径

StarryOS 的 TPU 输入 TDMA 为 1.114 ms，Linux 为 0.971 ms，相差 0.143 ms。绝对差值很小，可能同时受到定时器粒度、调度抖动和内存状态影响。从架构上看，Linux 官方 TPU 驱动使用 dma-buf 获取物理地址，并通过体系结构 DMA cache 同步接口维护一致性；StarryOS 当前采用 ION 零拷贝、coherent-uncached 映射，并将 cache ioctl 主要作为同步屏障。

Linux 的成熟 DMA 映射和运行时协作可能减少提交与同步开销，而 StarryOS 的专用 ABI、文件描述符解析、物理地址校验和完成等待仍会贡献少量固定成本。不过，0.143 ms 只占总时延约 0.27%，当前不应作为首要优化目标，除非多轮测试稳定复现并能由内核分段计时定位到具体步骤。

4)非缓存映射可能增加 CPU 后处理读取成本

StarryOS 后处理为 1.632 ms，Linux 为 0.449 ms，StarryOS 多 1.183 ms。两端的 `tpu.rs` 已核对为完全一致，均直接读取运行时管理的 FP32 输出张量，再执行阈值筛选、NMS 和坐标恢复。因此，该差异更可能来自运行环境，而不是后处理算法源代码。

一个值得重点验证的内核原因是输出张量的缓存属性。StarryOS 当前明确采用 coherent-uncached 映射策略；若 TPU 输出张量在 CPU 侧也通过非缓存映射读取，那么 CPU 扫描约

42,000 个 FP32 元素时会产生较高的内存访问开销。Linux 通常采用可缓存的用户映射，并在设备与 CPU 转换所有权时执行 DMA invalidate；完成一次同步后，CPU 可以利用数据缓存高速解析输出。该差异能够合理解释“TPU 硬件时间接近，但 StarryOS CPU 后处理明显更慢”的现象。

这一解释目前仍属于高可信度假设，而不是已完成的因果证明。要确认它，需要分别记录输出张量映射属性、cache invalidate 时间、纯顺序扫描带宽，以及不执行 NMS 时的张量遍历时间。

5) CPU 调度、频率和样本内容也会影响后处理

即使源代码相同，CPU 主频与动态调频策略、编译工具链、内存分配器、缓存冷热状态、系统中断负载和线程调度仍会影响 Rust 后处理时间。后处理耗时还与超过置信度阈值的候选框数量有关；不同测试过程中摄像头采到的目标位置和背景略有变化，可能使 NMS 的输入数量不同。

因此，下一轮应同时输出每帧原始候选数、阈值过滤后候选数、NMS 保留数、CPU 主频和上下文切换次数。在这些数据缺失时，不能只凭 1.632 ms 与 0.449 ms 就断言 StarryOS 的 CPU 或调度器整体慢于 Linux。

7.6 对置信度结果的解释

StarryOS 的平均置信度为 93.13%，Linux 为 93.62%，差值为 0.49 个百分点。两端使用同一模型，VPSS 输出格式和模型输入布局也已按同一标准实现，因此该差距更可能来自实际采样帧不同、球在画面中的大小与位置不同、曝光变化以及 letterbox 输入的逐像素微小差异，而不是操作系统直接改变了神经网络精度。

0.49 个百分点相对于当前场景波动较小，可以认为两端输出质量基本一致。但是，若要形成正式精度结论，仍需固定同一批输入图像或保存同一批 MJPEG 帧，在两个系统上离线回放，并计算 Precision、Recall、mAP50 和 mAP50-95。实时场景下的平均置信度只能作为一致性参考。

7.7 综合结论

本轮测试表明，StarryOS 通过专用 UVC/JPU 驱动、最新帧策略、ION 共享物理缓冲区以及紧凑的 JPU—VPSS 数据通路，在图像前端建立了约 4.173 ms 的优势。该优势抵消了 StarryOS 在 TPU 输入、TPU 推理、CPU 后处理和其他开销方面合计约 2.968 ms 的劣势，使其最终视觉总时延比 Linux 少约 1.206 ms，即快约 2.22%。

从系统架构角度看，StarryOS 的优势来自针对固定机器人工作负载进行跨层协同和路径裁剪；Linux 的优势则来自成熟的 DMA/cache 管理、TPU 官方驱动、等待队列和通用运行时生态。当前结果说明 StarryOS 已经在成功视觉帧的平均延迟上追平并略微超过 Linux，但领先幅度较小，尚不足以宣称在所有负载和统计口径下全面优于 Linux。

后续最有价值的优化方向不是继续压缩已经接近硬件极限的 VPSS，而是：第一，统一两端摄像头和 JPU 的时间戳边界；第二，拆分 TPU 的硬件执行、IRQ 恢复和运行时返回；第三，验证 TPU 输出张量的缓存属性并优化 CPU 后处理；第四，在相同图像回放、相同温度和相同系统负载下进行多轮测试，同时报告平均值、P50、P95、P99、失败率和任务墙钟。完成这些工作后，才能对两种系统的架构效率作出更严格、可复现的结论。

附录

当前正式链路的函数级调用过程

初始化阶段

```
main()
  ├─ 解析模型、摄像头、VPSS、电机和机械臂参数
  ├─ YoloModel::open(model_path)
  │    ├─ CVI_NN_RegisterModel()
  │    ├─ CVI_NN_GetInputOutputTensors()
  │    ├─ 检查输入 shape/format/pixel_format/aligned/quantization
  │    └─ 输出 AKARS_TPU_INPUT 契约
  ├─ VpssRgbPipeline::open(output_width=640, output_height=640)
  │    ├─ IonAllocation::allocate(source=2 MiB)
  │    ├─ IonAllocation::allocate(destination=3×640×640)
  │    ├─ open(/dev/cvi-usb-camera0)
  │    ├─ open(/dev/cvi-vpss0)
  │    ├─ CVI_CAMERA_IOCTL_INIT
  │    ├─ CVI_CAMERA_IOCTL_RESET_CAPTURE_STATS
  │    └─ CVI_CAMERA_IOCTL_START_ASYNC
  ├─ 打开 /dev/ttyS1 底盘电机
  ├─ 打开 /dev/ttyS2 机械臂控制器
  └─ run_tennis_hunter()
```

后台 UVC 采集阶段

```
async_capture_worker()
  ├─ 等待一帧在途背压条件满足
  ├─ capture_frame()
  │    ├─ uvc_capture_one_frame_profiled()
  │    │    ├─ USB/UVC 包接收
  │    │    ├─ FID/EOF 组帧
  │    │    └─ 返回 JPEG 与采集 profile
  │    ├─ 检查 SOI/EOI/最小长度
  │    ├─ 错误时复位 FID 连续性并有限重试
  │    └─ 更新 CaptureStats
  ├─ 写入非活动 latest-frame 缓冲
  ├─ 发布宽高、长度、时间戳与 sequence+1
  └─ 唤醒等待新序号的消费者
```

前台摄像头与 JPU 阶段

```
VpssRgbPipeline::next_inner()
  ├─ 构造 CameraIonFrameRequest(abi=2, last_sequence, source_ion_fd)
  ├─ ioctl(CVI_CAMERA_IOCTL_GET_LATEST_YUV_ION)
  │    └─ CviUsbCamera::latest_yuv_to_ion()
  │         ├─ wait_for_sequence(last_sequence)
  │         ├─ 解析并验证 source ION
  │         ├─ 复制小尺寸压缩 JPEG，释放 latest-frame 锁
  │         ├─ JpuDecoder::decode_planar_into()
```

```
  |          |     ├─ 复制 JPEG 到 JPU 位流缓冲
  |          |     ├─ 清零 1,024 字节尾部余量
  |          |     ├─ 编程 BBC/DPB/采样格式寄存器
  |          |     ├─ 启动并等待 JPU 完成
  |          |     └─ 输出 Y/Cb/Cr 到 source ION
  |          ├─ 失败时丢弃 sequence、复位并等待更新帧
  |          ├─ 返回三平面偏移/长度/步长、jpu_decode_us
  |          └─ mark_consumed(sequence)
  └─ 用户态校验 format=YUV422P、640×480 和平面范围
```

20.4 VPSS 阶段

```
VpssRgbPipeline::next_inner(execute_vpss=true)
  ├─ 构造 VpssRunYuv422pRgb(abi=1)
  |    ├─ source fd + Y/Cb/Cr offsets/strides
  |    ├─ crop=(0,0,640,480)
  |    ├─ content=(0,80,640,480)
  |    ├─ destination fd + R/G/B offsets/strides
  |    └─ border + timeout=100 ms
  ├─ ioctl(CVI_VPSS_IOCTL_RUN_YUV422P_RGB)
  |    ├─ 解析 ION 并校验范围
  |    ├─ 配置 YUV422P 输入地址、采样与步长
  |    ├─ 配置 BT.601 CSC、SC_V1 scaler、letterbox
  |    ├─ 配置 RGB planar ODMA
  |    ├─ 启动任务并记录 hardware_start
  |    ├─ 3 ms 有界完成观察 / IRQ 等待
  |    ├─ 记录 hardware_done 和调试状态
  |    └─ 返回 status/IRQ/timestamps
  ├─ split_vpss_timing()
  └─ 返回 destination physical address 与 RGB planar 视图
```

20.5 TPU 输入与模型前向阶段

```
YoloModel::infer_vpss_rgb_timed()
  ├─ 验证 destination 物理区间和张量契约
  ├─ CVI_NN_SetTensorWithAlignedFrames()
  |    ├─ libcviruntime 构造 TDMA-only 命令
  |    ├─ ioctl 提交 / TPU direct execution
  |    ├─ run_one(prepare → fire → IRQ → finish)
  |    └─ 将外部 RGB planar 导入模型输入
  ├─ CVI_NN_Forward()
  |    ├─ 提交 compute 命令流
  |    ├─ TDMA 搬运权重/特征图
  |    ├─ TIU 执行卷积、激活和检测头
  |    ├─ IRQ 唤醒提交线程
  |    └─ 返回输出张量
  └─ 分别记录 preprocess_us 与 forward_us
```

20.6 后处理与控制阶段

```
YoloModel::get_detections()
  ├─ CVI_NN_TensorPtr(output)
  ├─ FP32 直接只读切片
  ├─ parse_yolov8_output()
  |     ├─ 按通道读取 cx/cy/w/h/score
  |     ├─ 置信度阈值优先过滤
  |     └─ 构造少量有效 Detection
  ├─ nms(iou=0.5)
  └─ correct_yolo_boxes(640×640 → 640×480)

Robot::handle_detections()
  ├─ 选择最大面积网球框
  ├─ 计算 cx 和 area_ratio
  ├─ 搜索 / 左转 / 右转 / 前进 / 近球微调
  ├─ 连续两帧确认抓取条件
  └─ 进入机械臂抓取与持球状态
```

22.3 强制 JPU 输出 NV12

否决原因：摄像头 JPEG 是 4:2:2, JPU 原生三平面输出为 YUV422P; NV12 是 4:2:0 半平面。强制宣称 NV12 会造成采样和布局错误。正确方案是让 VPSS 官方支持 YUV422P 输入并完成 CSC。

22.4 普通模型输入直接物理地址绑定

回退原因：虽然源缓冲和张量哈希一致，但直接推理为零检测，说明绕过了运行时对内存类型、stride、命令补丁或缓存所有权的处理。正式路径恢复官方 `CVI_NN_SetTensorWithAlignedFrames()`。

22.5 P2 aligned 640 模型

回退原因：输入 TDMA 接近 0，但前向增加到约 80 ms，后处理增加到约 5 ms，共享内存显著膨胀，总时延约 134 ms。局部指标优化导致整体退化。

22.7 固定跳帧或降低处理帧数

固定跳帧可以降低平均资源占用，却不会降低被选中帧的 JPU、VPSS 和 TPU 单帧时延，还会增大控制反馈间隔。当前只在算法不需要 YOLO 的红车阶段跳过 VPSS/TPU，属于按需求裁剪，而不是盲目丢弃感知机会。

22.8 继续提高 compute 线程优先级

compute 的 97% 以上已经处于 `fire_to_irq` 硬件命令流。提高等待线程优先级只能影响 IRQ 后不足 1 ms 的恢复段，无法消除 39 ms 模型执行。继续修改调度可能破坏 USB 与控制线程公平性，收益有限。